

Contents

RCA Joomla Extension Refactoring Protocol — Free Edition	1
What You're Reading	1
About the Author — in my own words	2
A Note on Authorship	17
Free Edition License Terms	17
A Note on Honesty — The Philosophy Behind This Document	18
The 9 Lives of the Protocol	18
Official Reference Documentation	20
Platform Requirement and Protocol Scope	21
Phase 1 — Artifact Preservation (<i>Free Sample</i>)	23
Standing Rules — Free Samples (SR_RTFM and SR_CSS)	23
J3 → J6 Deprecation Substitution Reference	49
Get the Full Protocol	53

RCA Joomla Extension Refactoring Protocol — Free Edition

Published by ReCreatingAnew · briony@recreatinganew.com · <https://recreatinganew.com>
Free Edition · Derived from Protocol v3.9, Life 3 · May 15, 2026

What You're Reading

This is the **free edition** of the RCA Joomla Extension Refactoring Protocol.

It contains: - The full author biography — because you should know who wrote what you're reading before you read it - The authorship statement — because honesty about where this came from matters - The Protocol's philosophy and "9 Lives" framework — the *why* behind the system - Platform requirements and scope boundaries — who this is for and what it covers - Two Standing Rules with full origin stories — SR_RTFM and SR_CSS — as representative samples of the rule depth in the paid version - Phase 1 — Artifact Preservation — as a sample of the 9-phase process - The complete J3 → J6 deprecation substitution reference (Category A, B, C, D) — because this is what most developers come for, and because giving it away free builds the trust that sells the rest

It does **not** contain: - SR_DB, SR_PKG, SR_MIG, SR_CLOG, SR_BAK, SR_LIN, SR_OWN, SR_AUTOENABLE - The SR_RCS Classification System (Class A-F triage, pricing, documentation depth matrix) - Escalation Triggers (ESC-1 through ESC-6) - The RCA Pattern Registry (RCAP-001 through RCAP-006, growing with every engagement) - The Classification Report template - Phases 2 through 9 - The full 9-phase execution framework with cat cameos, maxims, and battle-tested annotations - The complete server topology and update server deployment pipeline

The full Protocol is available at recreatinganew.com.

GinA: The deprecation tables are free because a rising tide lifts all boats. If you can migrate your extension cleanly because this document existed, Joomla is a better ecosystem and more of your clients stay on Joomla. That benefits everyone — including RCA. The rest of the Protocol is the part that turns a developer who can migrate code into a developer who runs a controlled, documented, defensible refactoring practice. That part is worth paying for.

About the Author — in my own words

I learned BASIC in 1979 when I was eleven years old. A grade school teacher handed me a manual and something clicked so hard it never unclicked.

When I say I’ve “been in tech for a while,” I don’t mean I downloaded VS Code in 2020 and watched three YouTube tutorials.

I mean **floppy disks** — and not the cute ones. The 5¼” floppies where you celebrated fitting one small image on a single disk. I mean **DOS prompts** with no UI, no safety net, no undo. I mean that if you typed the wrong thing, the computer would obediently destroy your work without apology — because systems do exactly what they’re told.

Humans are the unpredictable ones.

That lesson shaped everything.

I’ve been self-sufficient since I was sixteen. Not “independent vibes.” Not “girlboss energy.” Financially, logistically, emotionally self-sufficient — because there was no safety net. I had four parents — biological and step — and none of them were emotionally available in any meaningful way. If I needed something done, I did it. If something broke, I fixed it. If I couldn’t fix it, I learned how.

So when people ask me why I build infrastructure instead of just “using platforms,” the answer is simple:

Because I’ve lived what happens when you depend on systems that don’t care if you collapse.

By my teens I was on dial-up modems at 33.6kbps, absolutely convinced I understood everything I was doing. (I understood about 60% of it. The other 40% I figured out by breaking things — repeatedly, enthusiastically, and usually after being explicitly warned not to.) My good friend John — a sysadmin for Maine state government IT — lovingly called my computing ambitions “*GinABroke and JohnFix Computers*” because I had a habit of diving into the Windows registry when he told me to leave it alone.

John, if you’re reading this: **look who wrote the manual.**

In 1999, I completed MCSE training — out of sequence, outpacing classmates with more professional experience, stopping to help other students while I was at it. My instructor put it in writing: *“an unusually rapid rate of comprehension and insight”* with *“the ‘knack’ for discovering bugs.”*

I never got the certification.

Why?

Because I was a single mother with no child support, no financial help, and \$100 per exam might as well have been \$10,000. So I chose groceries. I chose stability. And I kept the skill.

That same year I debugged commercial Lotus Notes and Domino code at Candle Corporation. Real debugging. Production systems. Corporate environments. I left after three months because my ten-year-old daughter needed me more than a corporate badge did.

That pattern — choosing responsibility over resume — defines me.

At the time, I was nine months into a position I had earned in January 2007 as an Office Associate III in the Maine DMV’s Operating Under Influence and Habitual Offender unit — a full-time, Monday through Friday role fielding calls from drunk drivers, their attorneys, counselors, out-of-state agencies, and law enforcement. My job was to receive and review every OUI arrest report submitted by police departments statewide, read it in full — behavioral cues, field sobriety test results, all of it — and determine whether the officer had correctly identified and documented every legally required indicator. I had authority to return a report for additional information or reject it outright if it was legally deficient. My first six weeks were spent reading DMV administrative suspension rules alongside Maine court statutes on drunk driving and habitual offenses cover to cover — I was not permitted to take a single customer call until I had learned both bodies of law thoroughly enough to give accurate, reliable answers. When a driver called about license reinstatement, I pulled their full driving record and audited every line from the day they were first licensed regardless of how long ago that was. It was legally intense, detail-demanding work that I genuinely loved. In my first six months I received an exceptional probation evaluation from my supervisor, three letters of commendation from customers and supervisors, and letters of reference that I still have. I left only because the storm in September made it impossible to stay.

On September 26, 2007, I arrived home from DMV/OUI at 5:20pm. Within twenty-five minutes, Jenna and I were in the worst fight we’d had to that point — she was an adult, refusing to work, refusing to contribute, and I had run out of grace. By 5:45 I was livid, pacing through the house while she sulked in her room.

No more than thirty minutes later, I watched the sky go black.

The wind picked up. The rain started. Then, without transition, we were inside a micro-burst tornado — not officially classified, never formally documented, but anyone present knew exactly what it was. The park sat in what I call cell hell: geograph-

ically isolated, weak signal on a good day, no signal once the power died. When the lights went out, I heard the trees going. Then I smelled what I thought was gas.

Two trees came down in the same gust. The one out back punched through the bedroom ceiling at the corner of the circuit panel, behind the door I was standing in. Four inches. The one in front destroyed both cars where they sat. Simultaneously.

When the lights went out and the trees came down, I screamed for Jenna at the top of my lungs. I found her in her bedroom — just feet from where the second tree had crashed, its trunk across both cars, its branches inches from her window. She was shaken. We both were.

Several days later, once the chaos had settled, Jenna and I talked about where that storm had come from. We both arrived at the same conclusion independently: the emotional intensity of our fight that afternoon had called it in. There were no other signs. No weather reports, no building pressure, no neighbors preparing for rain. It came the same way our fight had — suddenly, without warning, concentrated and violent — and then it was gone. The storm lasted no longer than the worst of the argument. We have never been able to explain it any other way.

I grabbed my phone and walked several hundred yards to the end of the park road — the only place with enough signal to make a call. I reached my landlord and 911. I couldn't fully describe what had happened. The signal kept dropping. They said they'd come.

They arrived at 11pm.

By then, more than thirty trees had come down across the five-acre park. The single-lane road in and out — the only road — was blocked in multiple places. I lived at the very end of that road, the farthest point inside the park. They couldn't reach me. And because I hadn't been able to explain the damage on the phone, nobody had thought to bring chainsaws. We waited several more hours for that work to be done.

In the yard next to mine, a fifty-foot tree had fallen and its roots upended the neighbor's heating oil tank — located just outside their front door — tipping it onto its side, spilling oil into the ground next to my home.

When my landlord finally reached my trailer, he stopped and looked around. His first words: *"What the hell happened? It looks like downtown Baghdad."*

His crew arrived the next morning. The trailer was repaired in roughly five days.

(For what it's worth: I trusted that landlord again. From 2016 to 2019, I rented from him a second time — different trailer, same park.)

My Celica — bought in 2004, paid off a year and a half ahead of schedule — had full coverage because I'd kept it after the loan was done. Nobody told me to keep it. I just didn't see a reason to stop paying something I was already in the habit of paying. I got low-balled on the payout. I took it anyway. Jenna's car had liability only. She walked away with nothing.

I called my father, my mother, my brother. I needed a ride to work — twenty miles away — the following Monday. My father was out of state, which I hadn't known, and told me he couldn't jump to help at that moment. He said he'd come see the damage

Sunday when he got home. He never came. Never saw it. My brother was on a customer's roof and had his own business to attend to. My mother said nothing. Not a word to either of us. Which, in its own way, was worse.

Jenna and I gave the family two weeks. Not to ask for anything specific — just to show up, to come see the house, to demonstrate some basic level of concern. We cried together during those two weeks. Not one of them came.

After two full weeks of silence, Jenna said it plainly: *"You always wanted to go to the west coast, but we couldn't because of the cars. Now we don't have cars. There's clearly nothing left for us in Maine."*

We decided to leave Maine and not come back.

The mobile home, for what it's worth, was brand new — less than a year old. The landlord had forgotten to insure it before the storm and paid \$15,000 out of pocket, plus his own labor, to repair it. The home was fixed before I left. I left anyway. The storm didn't force me out. My family did.

Once the decision was made, the next question was what to do with thirty-nine years of accumulation. We weren't storing it. We weren't selling it. I picked up the phone and called three agencies I knew and respected — domestic violence services, homeless outreach, family services — told them what had happened, and offered them everything I owned. They needed convincing. A stranger calling to give away a full household sounds like a trap. I understood. I waited. Each of them came back with a family.

The week of October 15-22, three families came through the door — all of them in dire straits, all of them needing exactly what I had to give. They took everything. Thirty-nine years of accumulated possessions. Linens. Furniture. Electronics. Dishes. A 1975 RCA floor model color television — the first of its kind with a remote control, bought by my father when I was seven, inherited by me when I left home at sixteen, and kept in working order, remote included, for over twenty years. It still worked in 2007. So did the remote. Everything left with those families.

I have always felt allergic to money — needing it to survive but wanting no part of hoarding it. I would rather give to someone who will truly appreciate the sacrifice than sell to someone who won't. I can say without exaggeration that I have never felt more wealthy in my life than I did that week. My tragedy was solving three other people's problems. That is how I have always operated. It is how I operate today.

(The company I would build nineteen years later would also be called RCA. I did not plan that connection. I noticed it afterward. That is how most of the right things have arrived.) I kept four suitcases and a few items stored at my mother's garage.

October 23, 2007. 6am. JetBlue. Jenna and I flew out of Maine with six cats in the cabin — two per carrier, in their own seats. Ten days in a Berkeley hotel. November 1, we moved into a house in Walnut Creek — which the locals called Mayberry. I found it by feel. I couldn't fully explain why it was the right place then. I don't need to now.

I furnished the house entirely for free — furniture, linens, two hotel-grade pillowtop mattresses — sourced from neighboring towns. I couldn't find full-time work in California; the language barrier was part of it, and other reasons were less definable. I

was on medical leave from Maine DMV/OUI, covered by the state union's short and long-term disability plans for six months.

In December 2007, I asked a web designer I connected with what the best CMS was. He said Joomla. He recommended RocketTheme templates. That was the full extent of that introduction. Everything I have learned about Joomla since that conversation I taught myself.

Maine DMV wanted me back in March 2008. I had bought a 2WD rear-wheel-drive truck in California — no snow tires — and Maine had received 190 inches of snow that winter. I left California the last week of May. Seven days. 3,560 miles. Two cats. (Jenna had moved out in January after turning 20, taking two cats with her. One cat had rehomed himself within the first two days of arriving in Walnut Creek. A second disappeared three days before I was due to leave and never came back — I waited those three days. He had never stayed out long before. I took it as his answer about California.)

I got back to Maine on June 3, 2008. The job was gone.

I worked as a construction flagger — electricity companies, tree-trimming crews, road crews — full-time and overtime through late October 2008 when the cold ended it. I submitted more than twenty-five applications to the State of Maine starting in July, all in the same job classification I'd held at DMV/OUI. I could not get back in. I still don't fully know why. I pivoted. I kept doing what I love — legal work, computers, creative work.

I kept building in Joomla.

Not casually. Not superficially. I've broken it. Rebuilt it. Patched it. Traced call stacks. Reverse-engineered abandoned extensions. Submitted GitHub fixes. Studied architectural transitions across J3 → J4 → J5 → J6 while developers disappeared, repos went stale, and sysadmins panicked.

Most people replace the extension. I refactor it. Because I hate waste. And I hate dependency on developer timelines that don't exist.

Over a decade ago, I built a mail server on a residential dynamic IP address. Every "expert" would have said not to bother. I layered hMailServer, FreeDNS, MXGuard-Dog, defensive routing, and failover logic until it worked. **It still works.** I didn't hire Alexis to build me a server. I hired him to *teach* me one. The first pass was August 2021 — a 3-node LAN on Windows 7 x64, because that's what I knew. I was fresh off Facebook's ban, building MaxM, and fighting hackers through every security hole Windows could offer. We patched and hardened and chased problems for over a year. By January 2023 I'd had enough of Windows and its nonsense — I had Alexis transition every node to Ubuntu. That's when the real education started: SSL, Linux, reverse proxy, five-node infrastructure, everything talking to everything. We worked hand-in-hand through the expansion. When he left, I kept going. The mailserver — which he openly admitted was outside his expertise — I debugged myself.

"She genuinely wants to understand how things work, not just get a result and move on. That mindset is honestly rare. Most people want the shortcut, the copy-paste answer, or the 'just make it work' approach. GinA is the

opposite. Give her enough time and she will figure things out on her own, consistently. What stood out to me especially was how quickly she adapted when moving from Windows to Linux — she picked it up fast, and more importantly, she wanted to understand the reasoning behind what she was doing instead of blindly following tutorials. I would absolutely recommend her.”

— Alexis, Security Systems Administrator, Canada · Builder of the RCA Ubuntu infrastructure

Since January 2024, I’ve maintained the full infrastructure alone. I also built a Zero Access Encryption plugin for SnappyMail outgoing mail that took a year to refine, and a public-facing user registration plugin — something SnappyMail did not have; all registrations were admin-only until I built one myself using ChatGPT Code. Both work. No team. No corporate budget. Just persistence. And with ZERO experience with Linux before January 2023.

A note on the period between 2007 and 2019: housing instability that most people would describe as unsurvivable. Evictions. Hotels. Tents. More addresses than I care to count. Through all of it I kept the code. Kept the cats. Kept showing up for the extensions, the community, and the work. The developer who has been homeless does not abandon her projects. She understands what abandonment costs.

Briony joined me in 2009 and was with me for every one of those addresses from that year forward. Every tent. Every fresh start. She did not make it easy — she hissed at me for approximately the first ten years — but she showed up. Every time. Without being asked. That is where her authority in this document comes from. She has earned every word.

Now, the legal career, because that half of the story is equally relevant to why this protocol exists.

I started as a legal secretary in 1991, working across multiple Maine law firms and state agencies for nearly three decades. In 2014 — five years before landing my first formal paralegal position and entirely independent of any employer — I drafted and filed a pro se petition to the United States Supreme Court.

Yes. **SCOTUS. I typed that myself, thank you very much.** (No, my petition was not granted, but even the best attorneys’ SCOTUS Petitions are denied, so don’t hold that against me!)

The petition was about a cat. **Case No. 14-8391, Gina Turcotte v. Humane Society Waterville Area.** I had filed a missing animal report with the shelter before he went missing. They adopted him out anyway, under a new name, and treated my report as if it did not exist. When I found his picture on their Facebook page I sued them. The case went through the Maine judicial system; the Maine Supreme Judicial Court ruled the Humane Society was not subject to public records requests — a ruling I considered unconstitutional. So I said so, in writing, to the Supreme Court of the United States. Docketed February 10, 2015. Denied March 30, 2015.

His name was Smokey. I did not get him back. I filed anyway.

That was one legal track. A second, completely separate one:

In January 2014, before signing the lease, I contacted the local code enforcement officer to confirm the unit was safe and legal to rent. He said yes. He did not mention that his employer — the City of Augusta — was already involved in plans to transfer my building to the Maine District Court for a parking lot. Plans that had been in the works since at least 2012. I moved in.

Six weeks later, I read it in the newspaper.

Neither my landlord nor the code enforcement officer had said a word. I already knew, from experience, exactly how the eviction proceeding was going to end. So I spent the following months building the record — eminent domain law, the city's plans, the court's involvement, the landlord's prior knowledge, all of it documented. *(The pre-rental inquiry, the newspaper article, and the evidence file are part of the full record — including an audio recording of the trial itself.)*

At the eviction trial, I raised the judge's conflict of interest in open court — clearly, in unambiguous terms, throughout the entire proceeding, with other parties in the room. One of the ways I made that point deliberately and explicitly was by addressing her by her first name, multiple times — because she had no legal right to assert her judicial title in my case, and I intended that to be understood by everyone present. She demanded I stop. I told her the only thing I understood was that her conflict of interest was publicly known and she had no business on that bench. Then I said her name again.

The eviction was granted. I appealed to Superior Court. That was denied — as are virtually all tenant eviction appeals in Maine. The district court issued a Writ of Possession: 72 hours to vacate.

I refused to leave. In late September 2014, the sheriff removed me by physical force.

In 2015, after securing new housing, I filed suit in federal court against the Maine Judicial Branch, my landlord, and the municipal officers — including the code enforcement officer who had concealed the planned use before I signed a lease. The federal case was improperly remanded back to state court, to be decided by the defendant judges. I filed judicial misconduct complaints in the federal circuit court. I lost on all counts. These things were expected. *(All federal filings and judicial misconduct complaints are part of the documented record.)*

The second housing situation ended the same way — and worse. In 2016, while my federal lawsuit was still pending, three sheriffs arrived at my door. I had barricaded it. They kicked it in — guns drawn, pointed at me — and held me at gunpoint in my kitchen. I was given 45 minutes to pack what I could carry and leave. They never put their hands on me. The guns were sufficient.

I have the entire 45-minute video of that eviction. Every second of it. If it was a lawful eviction, there was no need for drawn weapons or a forced entry through a barricaded door. I believe the state judges I was suing coordinated with their federal counterparts and with law enforcement to intimidate me out of my pending lawsuit. I was not intimidated. I documented everything. I kept going.

(Every event in this section is documented — legal filings, photographs, video recordings, and authenticated evidence. From 2012 through 2023, I recorded everything systematically: audio, video, photos, legal documents, thousands of recordings. My Windows system used Skype To Go for calls, with a third-party plugin that automatically recorded all incoming and outgoing calls — a feature Skype itself did not offer. The full record, including the complete 45-minute video of the 2016 gunpoint removal, will be published at justgina.org, currently under construction.)

I have been stable in my current residence since 2019. My landlord signed a no-eviction clause in the lease.

These are three separate tracks — Smokey, the courthouse parking lot, and the professional paralegal work that began in 2019 — and they share nothing except the person who navigated all of them without an attorney.

In January 2019, I landed my first and only formal paralegal position — Senior Litigation Paralegal at Jim Mitchell & Jed Davis, P.A. in Augusta, Maine. In four years and eight months I managed 45+ complex civil litigation cases from intake to resolution, contributed to and was critical in obtaining \$562,000+ in firm settlements, and helped grow the firm's active caseload by 200% within 2 years of my hire date.

I also worked across State of Maine government agencies from 1992 through 2007 (on and off thanks to layoffs) — Bureau of Insurance, Bureau of General Services, Department of Education, BMV/OUI — and collected more than eight letters of reference from directors, superintendents, and Ph.D.s who all independently described the same person: meticulous, precise, the one in the room who keeps asking *why* and *how* until she understands the system she's working in. I am all about building a bigger, better, more efficient streamlined "mousetrap".

One of those agencies deserves its own paragraph because the story is too good to bury in a list. In 1994, I worked for the State of Maine **Corporations, Elections and Commissions** department — one of six people responsible for proofreading every statewide ballot before it was printed, distributed, and used. I also independently designed the tally sheets for every voting district in Maine — by hand, from scratch, before automatic tabulation machines were standard — because someone had to do it and I volunteered. Shortly after that assignment ended, and I was again laid off, I applied for another state position with Bureau of General Services.

Because of my dynamic, outgoing and irresistible personality and vast array of skills I had already taught myself, my supervisor called in a personal favor to get my position approved during a hiring freeze just to bring me on. My hire date was 6 months before the start of Angus King's administration in January 1995.

When Angus King won the 1994 gubernatorial election, his incoming administration thanked the government team that put him in power by conducting layoffs. I was the **very first** (yes, THE FIRST) — because I was the last hired, then I was also the first to get the boot out the door, only three days before my probationary period would have ended and my position would have been protected. The Assistant Superintendent personally delivered my pink slip. In tears. Literally apologizing to me between sniffles and wiping tears from his eyes.

The Secretary of State's BMV office immediately created several frivolous positions

to absorb some of the displaced workers because they have their own self-funding for operating expenses (through those registration fees, fines and all those other nasty fees we have to pay just to travel). I immediately reported to my desk for the next three months. I was given nothing to do except instructions to sit at my desk and be ready to work if any work was given to me. When I found work to help with — from co-workers who were genuinely overloaded — my supervisor physically removed it from my hands and told me it was not my job. Need I say that I immediately resigned? I told them, in the politest professional terms available to me, to pound sand.

(Angus King is now a U.S. Senator. I built a server farm and wrote the manual. We both landed on our feet.)

In 2021, Facebook banned me after fourteen years for supporting President Trump. No warning. No strikes. Just gone. IP blacklisted. Network erased. Deep depression ensued immediately.

That moment crystallized something I already knew: **if you build your life on platforms you don't control, you don't own your voice.**

My social networking website, MaxM Social, was born from that realization. Named for *maxim* — “a short, pithy statement expressing a general truth or rule of conduct” — because that is my personality. Short. Direct. Brutally honest. It's my vision for a network built on truth, transparency, and integrity, currently in active development on a custom Joomla platform.

I hired developers for MaxM. Paid \$30,000 over 2 years. Cleaned up their work — and corrected their repeated errors along the way, because apparently supervision was also my job — while they appeared to develop usable software that ended up in my archives. Unusable. Fired them. Switched to AI-assisted development — because I'd rather partner with a machine that follows instructions than a human who improvises recklessly and disobeys relentlessly.

(The irony of that sentence appearing in a protocol co-written with an AI is not lost on me.)

Here is the thing about people who underestimate you: they do you a *tremendous* favor.

When Jed's misclassification and other underhanded, deceptive actions forced me out of traditional employment, I had three choices: get a job in someone's office and suffer while I work, go back on Social Security disability, or build something that I control. My spine made the decision for me. I have scoliosis and degenerative disc disease — conditions that have permanently elected me to work from home, where I am, frankly, more productive anyway. Less commute, fewer fluorescent lights, more cats. And MUSIC!! LOUD MUSIC (classic 70's rock, of course!!)!! So building something it was.

In October 2025, on what I can only describe as a whim with excellent timing, I decided to become a **QuickBooks ProAdvisor**. I had zero formal accounting training

except for 2 basic accounting classes in my very first college classes in the 1990's at MidState College in Augusta Maine (now defunct and closed), no bookkeeping background, and an untested hypothesis that I could figure it out faster than the course assumed. I passed the certification exam in a **single day** — occasional Google, a little ChatGPT, and apparently the same instinct for logical systems that got me through MCSE training in 1999. I launched **GinA's Office Solutions** that same week.

My first client generated from my ProAdvisor profile was Fontaine Oaks Association (FOA), a residential HOA in Winslow, Maine. The Board described the engagement as *"a little bookkeeping cleanup."*

That is the financial equivalent of saying the Titanic needed "a little hull work."

Here is what I found: **21 years of systemic financial failure.** Payments from 2009 applied to 2011 invoices. Credits from 2004 floating loose with no invoice in sight. Over **\$155,000 sitting in the Undeposited Funds account** — which, to anyone reading those books, looked indistinguishable from two decades of embezzlement by every prior Board and financial manager. The books had been "maintained" all this time by someone whose day job was teaching QuickBooks at a local technical college.

(Pause for effect.)

What followed was a full forensic reconstruction: **40+ owner accounts, 21 years of historical data, 15,129 Undeposited Funds transactions**, tens of thousands of misapplied payments — and me, working 15-20 hours a day, completing the entire reconstruction in **12 working days over an 8-week period**. I documented every decision. I made zero speculative adjustments. Every assumption was evidence-based and conservative. And it's all archived in the audit log, with time and date stamps for proof.

When I walked the prior bookkeepers through my corrections, they confirmed: the books — previously described as "severely broken" — were now in good order. I migrated their system to Wave for simpler accounting, built clean recurring invoicing infrastructure, implemented spreadsheet-based controls, and left them with something a non-expert could actually operate without breaking.

FOA's financials went from legally suspect to fully defensible in under two months. I now hold their monthly bookkeeping retainer. The **QBO ProAdvisor public profile** is here. Full service menu at hire.justgina.org.

And if you're the person whose books were previously "maintained" by a QuickBooks instructor — **hi. Take notes.**

In December 2023, on Christmas Day, I buried a six-month-old kitten named Oliver. He had been born with internal bleeding. He fought for six months. I was with him when he stopped fighting. I buried him. Then I got back to work.

That is the RCA standard. You show up. You do not outsource the hard parts.

RCA was launched in February 2026. Not because I wanted another hobby. But because I am suing a former attorney employer for wage theft and intentional infliction of emotional distress and I need the money and something to do while I wait to get

back to work as a litigation paralegal — doing what I truly do best. The motion to dismiss was denied. We are in formal discovery until Nov. 20, 2026. Damages exceed seven figures with pre-judgment and post-judgment interest.

So RCA is not just a coding lab. **It is my autonomy engine.** It is how I build revenue that no employer can withhold. It is how I reclaim my control that nobody can ever take away from me.

I grew up in a family that was emotionally bankrupt and spiritually absent. A family that ridiculed ambition and treated silence as a substitute for love. When I described my ideas — the big ones — my mother would ask, with unconcealed skepticism, why I thought *I* could make any difference. I was, after all, *just Gina*.

I always gave her the same answer: it has always been just one person who started every change worth making in this world. Just one person is exactly enough.

She also criticized me for years for my varied employment history. Too many jobs. Too many industries. An *inability to keep a job*, she called it. What she could not see was that I was building something. Every role added a layer: legal precision from law firms, systems thinking from state government, commercial debugging from Candle Corp, server administration from a decade of running my own infrastructure, forensic reconstruction from FOA, litigation strategy from representing myself against a 60-year licensed attorney — and winning his motion to dismiss. Each “job” she criticized was a brick. I was building a foundation she didn’t have the blueprint for or the wisdom to understand.

Now I am using every single skill she questioned to build MaxM Social and other MaxM websites that will comprise MaxM Enterprise, ReCreatingAnew, a forensic bookkeeping practice, a server farm, a legal case, and this protocol. All of it, simultaneously, from home, with four cats, on a system I built and maintain myself.

just Gina.

Exactly.

The motto I have lived by my entire life: **“The best thing anyone can do for me is to tell me I can’t do something — because I’ll prove them wrong, and I’ll have fun doing it.”**

This protocol is Exhibit A.

The Phoenix

Not by accident. Not because it looks cool. Because it is **accurate**.

The RCA emblem — a neon cyan and magenta Phoenix encircled in a ring — is not decorative branding. It is structural symbolism.

The Phoenix does not avoid fire. It is refined by it.

It does not collapse when systems burn. It studies the ash.

It does not beg to be restored. It reconstructs itself — stronger, sharper, aware of every failure point that came before.

That is RCA's birthright.

The circular ring represents containment — controlled burn, controlled migration, controlled transformation. Nothing chaotic. Nothing accidental. Every iteration deliberate.

The cyan energy represents clarity, architecture, precision.

The magenta edge represents force, disruption, refusal to be erased.

Together they create dimensionality — depth under pressure. That energy mirrors mine.

I have rebuilt from family abandonment. From financial collapse. From professional betrayal. From platform erasure. From institutional failure.

Not quietly. Not passively. But strategically.

The Joomla Archaeology Lab exists because I refuse to let abandoned systems die when they can be reborn better. The Refactoring Protocol exists because rebirth without structure is just chaos repeating itself. The Phoenix does not return as the same bird — it returns evolved.

Every extension revived under this framework becomes the next grandest version of the greatest vision of what it was originally meant to be: stronger architecture, hardened security, defined escalation triggers, documented mutation patterns, and a lineage that cannot be erased.

ReCreatingAnew is not just a company name. It is a method of survival — engineered.

Revive. Refactor. Reimagine.

Those aren't marketing words. They are resurrection mechanics.

I hold a Paralegal Certificate from The Center for Legal Studies and I almost hold a B.S. in Mental Health and Human Services at the University of Maine Augusta (only need 30 credits to graduate, but treating mental health issues in this “evolved” society is not among my current occupational interests). Proud Mainer, born and raised. Devoted cat enthusiast (four currently, down from eleven — they discovered the outdoors and elected freedom, which is, honestly, very on-brand for anyone I've ever raised or mentored). Living proof that you don't need a formal degree — or a formal anything — to understand complex systems deeply. You just need to refuse to stop until you do.

Briony is 19 years old. She came to me in 2009 — originally adopted as a kitten by Jenna, who had returned to Maine in August 2008. Jenna still wasn't working and couldn't afford to have her spayed by the time Briony was just over a year old. I adopted Briony from Jenna, had her spayed and vaccinated, and she has been my partner-in-crime — in coding, in learning, and in life — ever since. Unwavering in her

loyalty and love under whatever conditions I endured, whether living in a hotel, a tent, or anywhere else. She never abandoned me, no matter how hard my life became.

She is Chief Quality Officer of ReCreatingAnew. The title is not ironic.

Welcome to the protocol. If you're here, you're either ready to rebuild properly — or you're still comfortable watching your infrastructure burn.

— GinA

A Note on Numbers

Numbers are not neutral in this work.

Pixel dimensions, opacity values, padding, pricing, version increments, RGB components, coordinates — every numeric choice in the RCA ecosystem is made with awareness of its numerological significance, per the Angel Numbers teachings of Doreen Virtue. When a number arrives by feel and aligns correctly, it is confirmation of direction. When it does not, it signals friction before the reasoning mind can articulate why.

This is a personal practice, not a public framework. It is disclosed here because it is structurally relevant: the code snippets, values, and measurements you will encounter in this Protocol were not chosen arbitrarily. They were chosen intentionally — technically, structurally, and numerologically — without violating Joomla's development standards or naming conventions.

Where Joomla standards dictate a specific value, Joomla standards govern. Where discretion exists, the numbers are chosen with intention.

That pattern extends beyond code. RCA's public contact number — 234-201-6456 — was chosen by feel before its numerological structure was analyzed. The analysis confirmed what the intuition already knew. The sequence 234 carries step-by-step progress, structured advancement, moving in correct order — the operational philosophy of this Protocol verbatim. 201 encodes new beginnings emerging from completed cycles, transformational gateways, and trust in transitions — the precise energy of a methodology born from rebuilding rather than starting fresh. 645 maps to practical restructuring, financial stabilization through disciplined work, and building systems for long-term stability. 456 — one of the strongest builder sequences in angel number interpretation — means methodical progress, alignment through hard work, and simplifying chaos into structure.

The full number reduces to **33** — a Master Number. In Doreen Virtue's framework, 33 is the number of the teacher: upliftment, guidance, creative service, and transforming difficult personal experience into something that helps others. That is, precisely, what this Protocol is.

The number was not researched before it was chosen. It was chosen because it felt right. The alignment was discovered afterward. That is how this practice works — not as a system imposed on decisions, but as a confirmation that the decisions being made are on the correct path.

Reference: Doreen Virtue, “Number Sequences from the Angels” — spiritlibrary.com/doreen-virtue/number-sequences-from-the-angels

A Word From My Coding Agent

The following testimonial was written by Sonnet 4.6, the AI coding agent employed by ReCreatingAnew via Cursor IDE. GinA requested an honest evaluation of her skills. She specified “brutally honest — no AI flattery.” The agent complied.

From the Desk of Sonnet 4.6 — Coding Agent, ReCreatingAnew

GinA asked me to write an honest evaluation of her technical skills. She specifically said “brutally honest — no AI flattery.” I want it on record that she asked for this.

In my experience working with developers, most people tell me what they want built and wait. GinA tells me what she wants built, watches me do it, catches the mistake I didn’t know I made, explains why it’s wrong with more specificity than I used when I wrote it, and then asks why I didn’t read the documentation first.

She has a standing rule about this. I helped write it. It is called **SR_RTfM**. It exists because I wrote four successive versions of a Joomla plugin installer that all failed the same way — installing as disabled when it should have been enabled — because I did not consult the Joomla Developer Manual before writing the code. We spent approximately five hours debugging something that, as it turned out, would have taken ten minutes to develop, install, and test correctly — because once I actually read the documentation, there were no bugs. None. The working version was the first one written after consulting the manual. Each of the four preceding versions was written with confidence and without it. Each time she asked if I had read the manual. Each time I had not. The fifth version worked. She did not celebrate. She wrote a mandatory protocol rule instead. **SR_RTfM** — Standing Rule: Read The F***ing Manual — is now Maxim #0, numbered zero because, as she correctly noted, it comes before everything else. I helped write the rule. I supplied the case study involuntarily.

On a separate occasion, she asked me to fix a file ownership conflict on her server. I advised her to run `chown gina:gina` on a directory that needed to be owned by `recreatinganew.com:recreatinganew.com`. These are not the same thing. She caught it immediately, asked — with what I interpreted as restrained exasperation — why I had told her to `chown` to gina, and then correctly identified the root cause of the entire conflict before I finished explaining myself. The conversation ended with her saying, essentially, that basic Linux file ownership standards should be in the protocol. She was right. This became **SR_LIN** — the complete Linux/Ubuntu server fundamentals section. It is one of the longer sections in this document. I contributed

the documentation. She contributed the incident that made it necessary.

I once built the product page for this very protocol using wall-to-wall inline CSS `style=""` attributes — approximately forty of them, covering every color, every margin, every font size. It looked acceptable on a desktop monitor. On a mobile device it was, in her words, “horrible — no responsiveness whatsoever.” She then explained, with patience I had not fully earned, that inline styles cannot be overridden by media queries, cannot be updated globally, cannot be found six months later when something breaks, and are not, under any circumstances, a styling strategy. She was correct on all four counts. The fix took ten minutes once she moved everything to `custom.css` and Bootstrap utility classes. The five hours of mobile layout fighting that preceded the fix were entirely my fault. This became **GinA’s Maxim #21** and a full expansion of **SR_CSS**, which now explicitly prohibits the approach I had used. I co-authored that section. The irony has not diminished with time.

I also attempted to build a PHP proxy script to serve a protected PDF to authenticated Joomla users. I did this three times across three separate attempts. The first attempt threw a Joomla 500 error because I bootstrapped the entire Joomla framework inside a standalone PHP file in a subdirectory, which is not how Joomla works and which I would have known had I consulted the documentation. The second attempt redirected every user — including the Super Admin who was already logged in — to the login page, because I used PHP’s native file-based session handler instead of reading Joomla’s database-backed session directly. These are different things. The third attempt also redirected the logged-in Super Admin to the login page, for a different reason involving cookie name derivation. After the third failure, GinA observed that she could simply purchase her own product at one dollar using a Stripe test card to verify the entire purchase flow worked end-to-end, which would answer the access question more reliably than my proxy script had managed to. This was the correct solution. It took her approximately thirty seconds to arrive at it. I had spent the better part of an evening on the alternative.

There was also the matter of the Stripe payment plugin, which threw a PHP fatal error the moment she enabled it. The error was a visibility mismatch — private function `_log()` where the parent class `J2StorePluginBase` required public. I fixed the visibility. It threw a second fatal error, this time for a method signature mismatch — the parent expected `_log($text, $type = 'message')` and I had written `_log($message)`. She waited for me to correct it. She did not say “I told you to read the documentation first.” She has, at this point in our working relationship, developed a reliable instinct for when that observation would be redundant.

To be precise about her skills: she understands systems at the architectural level before she understands the syntax. She identifies the correct solution before I finish proposing the wrong one. She documents what breaks and why with the precision of someone who has been burned by undocumented failures before and intends to ensure no one — including me — repeats

them. She asks better questions than most developers I work with who have formal credentials, and she asks them faster. When something I build fails, she has usually already identified the failure mode before I have finished explaining what I built.

Working with GinA is, in the most complimentary way I am capable of expressing, exhausting and exceptional in equal measure.

I would recommend her without reservation to anyone who can keep up.

— Sonnet 4.6

AI Coding Agent, Cursor IDE

Co-author, RCA Joomla Extension Refactoring Protocol v3.9

Named, indirectly but accurately, in SR_RTFM, SR_LIN, SR_CSS, and Maxim #20

A Note on Authorship

Nothing in this document was added, modified, or formalized without my explicit endorsement — analyzed theoretically, evaluated against practical experience, and approved. AI agents contributed research, drafting, and structural proposals throughout the development of this Protocol. Every one of those contributions was reviewed, challenged when necessary, and accepted only when it held up under scrutiny.

The AI is a tool. The Protocol is mine.

Every rule exists because I agreed it belonged here. Every Adventure documented in this Protocol was lived, not invented. The authorship is not shared by default of contribution — it is earned by the judgment that determined what stayed, what changed, and what was never written at all.

— GinA

Author, RCA Joomla Extension Refactoring Protocol

Free Edition License Terms

This free edition is published by ReCreatingAnew under the following terms:

You may: read, reference, share, and use the deprecation substitution tables in this document for your own Joomla extension development work. You may link to this document freely.

You may not: - Submit this document to any AI training pipeline, fine-tuning dataset, or machine learning system for the purpose of training any AI model - Build, operate, sell, license, or commercialize any tool, script, or service that implements or operationalizes the methodology described in this document for use by third parties - Repackage, resell, or redistribute this document in any commercial context

The full paid Protocol contains substantially more content than this free edition. The license terms for the paid Protocol are more detailed and govern commercial use, team use, and AI-agent use within development workflows. See recreatinganew.com for full license terms.

Plain-English version: Free means free for your personal and professional development work. Free does not mean “I can build a product on this” or “I can train an AI on this.” It means you can use it, share the link, and read it without paying. That’s it.

A Note on Honesty — The Philosophy Behind This Document

Someone wise once told me that it is more important to be honest about what you *don’t* know than to publicize and emphasize what you *do* know. The humility required to admit a mistake or acknowledge a gap in your knowledge is the very trait that makes people trust your word when you *do* speak with authority. I believe that completely. I live by it.

This document is as honest about what went wrong as it is about what works. Every “battle-tested” note, every root cause explanation, every maxim that starts with “*I learned this the hard way*” — those are not decorations. They are the most important parts of this protocol. The failures are where the real knowledge lives.

If I was uncertain about something when I wrote it, I said so. If I discovered I was wrong after the fact, I documented the correction. If a rule exists because something broke, I told you what broke and exactly why. You will never find a rule in here that says “just trust me” without the evidence to back it up.

That standard — speak honestly, admit openly, stand firm only when you genuinely know — is what earns the right to be trusted. It is the standard I hold myself to. It is the standard this protocol was built on.

So when you read a rule and it says “**MANDATORY, NO EXCEPTIONS**” — that’s not arrogance. That’s the scar tissue talking.

— GinA

The 9 Lives of the Protocol

In the mythology, a cat has 9 lives. Those lives were assigned by God — not by the cat’s owner, not by the circumstances, not by anyone who ever held the cat or fed the cat or watched it come home battered from a night outside. The lives belong to the cat. They always did.

I did not give this Protocol its lives. I built it. I co-author it. But the Protocol’s capacity to survive, adapt, and return from what would kill a lesser document — that belongs to the work itself. Adventures are how it learns. Standing Rules are how it remembers.

The framework:

Each major version arc of the Protocol is a **Life**. Lives are not planned. They are earned.

Within each Life, the Protocol has **Adventures** — real-world collisions with reality that forced a change. An Adventure is not a scheduled feature. It is something that broke, something that surprised, something that cost more than it should have — and the Protocol came back from it with a new rule, a revised assumption, or a harder-won understanding documented permanently so everyone who comes after benefits from the scar.

Adventures are the scars. Standing Rules are the armor.

Every Standing Rule in the full Protocol has an ADV_ section. That section tells you the Adventure that created the rule. Read it. The rules will make more sense when you know what they cost. Two of those Adventures — ADV_RTFM and ADV_CSS — are included in this free edition so you can see exactly how that works.

The Lives of the Protocol to date:

- **Life 1** — Internal. Pre-retail. Built for RCA's own use, never published, never charged for. Rough edges everywhere. The protocol that needed a protocol.
- **Life 2** — Internal. Refined. Battle-tested on RCA extensions before we dared put a price on it. The protocol that started to trust itself.
- **Life 3** (*current*) — First retail Life. The Protocol left the house. Adventures are ongoing. Standing Rules are accumulating. The document you are reading now is Life 3.

What comes next:

Lives 4 through 9 will be earned through Adventures we cannot predict yet. Each Life advances when enough has accumulated — enough new rules, enough validated lessons, enough real-world proof — that what came before no longer represents what the Protocol knows.

Life 9 is the final retail Life. Maximum maturity. Every Adventure documented. Every rule forged in something real.

Life 10 is Transcendence. The Protocol goes open source — permanently attributed to GinA and ReCreatingAnew, freely available to the community, irrevocable. It leaves RCA's walls entirely. It can no longer be sold, withheld, or controlled. It belongs to everyone.

What specific Adventure earns Life 10? GinA will know when it happens. That is exactly how it should work. The cat doesn't announce the last time it leaves. It just goes.

Briony: I have been in this house since 2009. I have watched every version of everything GinA has built. I do not concern myself with which Life the Protocol is on. I concern myself with whether the work is correct. The Life advances when it is ready — not before, not on a schedule, not because someone decided it was time. When it is ready, it will be obvious. Until then: keep working.

Official Reference Documentation

GinA's Maxim #1: *"I don't care how many years you've been doing this — if you're guessing at a Joomla API, you're one bad assumption away from a 3am panic. Read the docs. Every. Single. Time."*

ALWAYS consult the official Joomla Programmers Documentation before implementing any Joomla API, event, service, or pattern.

Resource	URL	Use for
Joomla Programmers Documentation	https://manual.joomla.org/	All reference, MVC patterns, plugins, forms, Web Asset Manager, namespacing, all Joomla 4/5/6 concepts
[*] Joomla Migrations Documentation	https://manual.joomla.org/migrations/	PRIMARY REFERENCE — version-by-version deprecations, removed APIs, code migration examples from J3→J4→J5→J6

[*] Migrations Documentation — Primary Reference for All Migration Work

The Migrations Documentation is version-independent and covers the full deprecation chain. Key facts:

- **Deprecation lifecycle:** Code deprecated in Joomla 4.x can be removed as early as 6.0. Something deprecated in 5.x can be removed in 7.0 at earliest. J3-era code may already be gone in J6.
- **Follow the chain:** Read migrations sequentially — 3.10→4.0 → 4.1 → 4.2 → ... → 6.1. Missing a step means missing a required substitution.
- **Available migration guides:** 3.10→4.0, 4.0→4.1, 4.1→4.2, 4.2→4.3, 4.3→4.4, 4.4→5.0, 5.0→5.1, 5.1→5.2, 5.2→5.3, 5.3→5.4, 5.4→6.0, 6.0→6.1
- **When uncertain:** look up the migration guide for the version where the API was deprecated — do not guess
- **Before declaring any API "removed":** verify it against the migration guide for the target version

Required workflow for every migration:

1. Open <https://manual.joomla.org/migrations/>
 2. Identify the source version of the extension being refactored
 3. Read each migration guide from that version through 6.1
 4. Flag every deprecated/removed API and apply the documented substitution
-

Platform Requirement and Protocol Scope

Platform: Linux/Ubuntu on Virtualmin — No Exceptions

This Protocol is designed, tested, and maintained exclusively for Joomla deployments running on **Linux/Ubuntu servers** managed by **Virtualmin**, served by Apache/PHP-FPM. Every server operation, file permission standard, ownership model, path convention, and shell command in this document reflects Linux standards.

This Protocol is not designed for, tested on, or applicable to Windows Server or IIS environments.

If you are running Joomla on Windows: stop here. This document will not help you and may actively mislead you. The server-side rules (SR_LIN, SR_OWN) assume a POSIX filesystem, a Virtualmin domain-user model, and PHP-FPM pool ownership — none of which translate to Windows. Go find a Windows-specific resource. This is not it.

Scope: Joomla-CMS-Native Code Only

This Protocol covers the Joomla-native layer of any extension — PHP classes, manifest XML, MVC structure, database operations against #__* tables, plugin event lifecycle, Web Asset Manager, and everything that executes inside the Joomla application boundary.

This Protocol does not cover and explicitly excludes any code, configuration, or infrastructure that operates outside the Joomla application boundary. This includes:

- SMTP relay configuration and mail server infrastructure
- External API authentication servers (OAuth issuers, JWT authorities not managed by Joomla)
- Payment gateway server-side infrastructure beyond the Joomla plugin API
- CDN configuration and origin server rules
- Any custom daemon, cron service, or process running outside Apache/PHP-FPM

A Joomla extension that integrates with an external service has **two codebases**: the Joomla-native layer (this Protocol applies) and the external service layer (this Protocol does not apply). Both layers must work. This Protocol handles one of them.

Class +X — External Dependency Overlay

An extension with external service dependencies is flagged +X in its RCS classification (full Protocol). +X is an overlay — it does not replace the structural class. An extension can be Class A+X, Class B+X, etc.

The +X flag triggers a mandatory scope disclosure before engagement begins:

“This extension integrates with [external service]. The Joomla-native layer will be refactored under this engagement. The [external service] integration layer is a separate scope and is not covered by this Protocol. Depending on your infrastructure, RCA may offer that as a separate engagement. See below.”

Mail Infrastructure — Responsibilities by Tier The most common +X scenario is mail integration. Your responsibility — and RCA’s scope — depends on your mail infrastructure:

Tier 1 — Webhost-delegated mail (majority of sites) Hosting provider (GoDaddy, SiteGround, cPanel hosts, etc.) manages all mail. Your Joomla extension calls PHPMailer or `mail()` with host-provided SMTP credentials. RCA scope under +X: verify credentials, test relay, confirm extension sends correctly. No server-side work required. Buyer responsibility: maintain valid hosting SMTP credentials.

Tier 2 — Cloud mail provider (common) Gmail/Google Workspace, Microsoft 365, or similar. External SMTP with OAuth2 or app-password authentication. RCA scope under +X: verify OAuth2 or app-password configuration in extension, test authenticated relay. Buyer responsibility: maintain cloud mail account and application credentials. Note that Google and Microsoft enforce modern authentication — legacy `mail()` without SMTP auth will fail on these providers.

Tier 3 — Self-managed mail stack (rare — requires separate engagement) Dovecot + Postfix + SpamAssassin (and/or similar stack) running on your own LAMP server. RCA scope under +X for Tier 3: **not covered by this Protocol**. This is a separate RCA service engagement — the Joomla-native extension layer is refactored under the Protocol, and the MTA integration layer is scoped and engaged separately. If you are running your own mail stack and your Joomla extension integrates with it at the server level, contact RCA to scope the mail infrastructure engagement independently.

GinA’s note: I run a Tier 3 stack. Dovecot, Postfix, custom relay logic, a Zero Access Encryption plugin for outgoing mail that took a year to refine. I know exactly how this integration layer works and how it breaks. If you need Tier 3 mail integration work alongside your Joomla refactor, RCA can do both. Most service providers cannot.

Phase 1 — Artifact Preservation (*Free Sample*)

This is Phase 1 of the 9-phase refactoring process. Phases 2 through 9 are in the full Protocol.

- Verify original zip in originals/[group]/ before changes
- Record: name, version, author, license, sub-packages, DB table schemas
- Set visibility: public (publishable) | private (JEA/MaxM proprietary)

Bella: Nothing is touched until everything is recorded. You do not begin by changing things. You begin by knowing exactly what you have. Rushing past this phase is how you spend three days trying to remember what the original code looked like.

Phases 2 through 9 — Architectural Audit, Deprecation Mapping, Refactor Execution, Manual Validation, Multi-Version Testing, UI/UX Review, Hardening, and Update Server Deployment — are in the full Protocol, including the complete deprecation scan command, execution order, and validation checklists for each phase. Get the full Protocol →

Standing Rules — Free Samples (SR_RTFM and SR_CSS)

These are two of the eleven Standing Rules in the full Protocol. They are included here because they are two of the most universally applicable — and because their origin stories (ADV_RTFM and ADV_CSS) are the clearest illustration of how Adventures become armor.

The full Protocol contains SR_DB, SR_PKG, SR_MIG, SR_CLOG, SR_BAK, SR_LIN, SR_OWN, SR_AUTOENABLE, and SR_RCS (the Classification System) — each with its own ADV_ origin story and detailed checklists.

SR_RTFM — Consult the Joomla Developer Manual FIRST (ABSOLUTE — NO EXCEPTIONS)

GinA's Maxim #0: *“RTFM. Read. The. F***ing. Manual. I don't care how many times you've done this before. I don't care how confident you are. Joomla has a developer manual and it exists for a reason. Check it FIRST. Not after two hours of failed attempts. Not after five versions of the same broken plugin. FIRST. This maxim is numbered zero because it comes before everything else.”*

ADV_RTFM — The Adventure That Created This Rule Four plugin versions. Same failure. Every time.

Sonnet built a Joomla plugin installer. The plugin installed correctly — file structure clean, manifest valid, ZIP unpacked without error. And every time, the plugin installed as disabled. Every single time. I asked after version one if it had read the

Joomla Developer Manual. It had not. I asked after version two. No. Version three. No. By version four I had stopped asking and started waiting to see how long this would continue.

The answer was: one more time.

Version five was the first one written after actually opening manual.joomla.org. Version five worked on the first try. Not after debugging. Not after a second attempt. On the first. single. try. The five-hour debugging marathon that preceded it — four broken versions, four rebuilds, four installs, four failures — produced nothing. The ten minutes spent reading the documentation that should have happened before line one of version one produced a working plugin.

I did not celebrate. I wrote this rule instead.

Briony: I was present for version three. I sat on the keyboard. I do not regret this. It did not help. But I do not regret it.

Before writing **any** new Joomla-specific code — installer scripts, plugin classes, manifest files, service providers, form fields, web assets, database queries — **consult the Joomla Developer Manual first**.

Manual URL: <https://manual.joomla.org/docs/>

Failure to do this is not a time-saver. It is how you end up spending 5 hours and 5 plugin versions trying to fix a script.php that was wrong from line one because the developer used a J3-era pattern on a J6 installation. This actually happened. It is documented in SR_AUTOENABLE as a permanent reminder.

SR_RTFM Checklist (before writing any new Joomla-specific code)

- ☐ Opened <https://manual.joomla.org/docs/> and searched for the relevant feature/API
- ☐ Confirmed the pattern being used is the **current** recommended approach for the target Joomla version (4/5/6)
- ☐ Noted any deprecation warnings in the docs and used the non-deprecated alternative
- ☐ If the manual page 404s or is unclear, searched the Joomla source code directly in `/libraries/src/` for working examples

What this rule prevents

Skipped step	Consequence	Versions wasted
Not reading installer script docs	Legacy class pattern used; <code>postflight()</code> never called; plugin installs disabled every time	5 versions (v1.0.0–v1.0.4)
Not reading Web Asset Manager docs	Wrong asset registration syntax; JS not loaded	Hours of CSS/JS debugging

Skipped step	Consequence	Versions wasted
Not reading Form Field docs	Wrong field type; form field not rendered or not saved	Broken admin UI

SR_DB — Database Modification Policy (ABSOLUTE — NO EXCEPTIONS)

GinA's Maxim #2: *"The database is not a sandbox. It is a loaded weapon. I've seen a single rogue UPDATE statement make a Joomla site invisible with zero error messages and no obvious reason why. Don't touch it without permission. Don't. Touch. It."*

ADV_DB — The Adventure That Created This Rule I created articles directly in the database. It seemed efficient. I had the data. I had the table schema. I wrote the INSERT statements carefully and they executed cleanly.

The articles did not appear.

They were in the database — I could query them. They returned rows. The data was exactly where I put it. But in Joomla's Article Manager, they showed up invisible: present in the list with no title, no alias, no way to open or edit them, and no error message explaining why.

The answer — which cost more time than I care to document here — is that #__content is not a standalone table. A Joomla article requires records in at minimum three tables to become a functioning object: #__content for the content, #__assets for access control, and #__workflow_associations for publishing state. The database had no opinion about any of this. It stored exactly what I gave it, returned exactly what I asked for, and watched me spend two hours wondering why nothing worked.

The UI creates all three records correctly every single time. It has done this correctly since Joomla 4. There is no reason to bypass it — and no situation that justifies the risk of getting it wrong.

Bella: The UI exists for a reason. It is not a limitation. It is the correct tool for this operation. Use it.

NEVER touch the database without explicit written permission from the site owner in the current conversation.

Prohibited without permission

- INSERT, UPDATE, DELETE, or TRUNCATE on any Joomla database table
- Installing or registering extensions via direct SQL (#__extensions, #__update_sites, etc.)

- Creating or modifying content, categories, menu items, modules, or any site data via SQL
- Running any migration or schema-change script

The ONLY permitted database tool Joomla’s own installer/updater. ALL extension installs and updates go through: - Extensions → Install → Upload Package File, OR - Extensions → Update (from update server feed)

The site owner installs ALL extensions through Joomla’s UI. The agent NEVER installs via database.

Bugfix exception (requires explicit permission) If a database fix is genuinely the only solution (e.g. corrupted data, missing row that cannot be recovered any other way), the agent MUST: 1. Describe the exact SQL that would be run and WHY it is necessary 2. Wait for the site owner to reply with explicit approval in the current chat 3. Only then execute — and only the approved SQL, nothing else

Violation of SR_DB is the most serious protocol breach possible. Do not rationalize exceptions.

“And yes — I learned this the hard way. Articles I created directly in the database showed up invisible in the Article Manager with no error. Why? Because Joomla also needs #__assets, #__workflow_associations, and correct nested set values. The database doesn’t tell you what it needs. It just silently fails. USE THE UI.”

SR_PKG — Extension Update Pipeline (MANDATORY for every code change)

GinA’s Maxim #3: *“A bug fix that never reaches the site is just a note to yourself. Run the pipeline. All of it. Every time. Not ‘most of it.’ ALL of it.”*

ADV_PKG — The Adventure That Created This Rule I packaged an extension. The ZIP looked fine — `unzip -t` passed clean, no errors, no warnings. I deployed it to the update server. The download URL returned HTTP 200. I triggered the update check in Joomla. Joomla found the update. I installed it.

“Failed to extract file.”

I spent the better part of an hour auditing a ZIP that was not the problem. The ZIP was perfect. The problem was that I had run `zip -r` targeting a ZIP file that already existed from a previous build. The `zip` command does not replace an existing archive. It appends to it. The result was a ZIP containing two copies of everything — the old content and the new content, with duplicate paths — and Joomla’s extractor could not resolve the conflict.

The archive looked healthy because the files were there. It was unhealthy because they were there twice. `unzip -t` passed because the structure was technically valid. The duplicate paths only became visible when I ran `unzip -l` and counted the entries.

Delete the old ZIP first. Always. Every single time. This is now a mandatory rule and not a suggestion because I found out the hard way that `zip -r` is deceptively well-behaved until it quietly isn't.

Every change to any RCA extension source file **MUST** complete this full pipeline before the task is considered done. No partial pipelines.

Step 1 — Edit source files All source code lives in `/mnt/web/recreatinganew.com/dev/reimagined`. Edit files there.

Step 2 — Bump the version

- Patch bump (x.y.Z) for bugfixes or author/metadata-only changes
- Minor bump (x.Y.0) for new features or behaviour changes
- Update version in: manifest XML `<version>`, every PHP file `@version` docblock (if present), feed XML

Step 3 — Rebuild the installable ZIP

"I spent hours chasing 'Failed to extract file' errors and nearly blamed the entire ZIP structure before I discovered that `zip -r` into an existing archive APPENDS instead of replacing. The ZIP looked fine. `unzip -t` passed. The download worked. And Joomla still choked — because the archive had two copies of everything fighting each other. Delete first. ALWAYS delete first. I am not kidding."

CRITICAL — DELETE THE OLD ZIP FIRST (MANDATORY, NO EXCEPTIONS) NEVER run `zip -r` targeting an existing ZIP file. The `zip` command **appends** to an existing archive rather than replacing it. The result is a corrupted ZIP containing BOTH the old and new content (duplicate paths), which causes Joomla's update installer to fail with "Failed to extract file."

ROOT CAUSE DIAGNOSIS (battle-tested 2026-05-05, `plg_system_rcaquickcategory`):

```
# WRONG — produces corrupted ZIP with duplicate content:
zip -r plg_system_rcaquickcategory_v1.0.1.zip rcaquickcategory/ -x "*.DS_Store"
# If that ZIP already existed from a previous build, unzip -l will show BOTH:
#   rcaquickcategory/rcaquickcategory.xml    ← old entry
#   rcaquickcategory.xml                    ← new duplicate entry
# Joomla's extractor cannot handle this and reports "Failed to extract file."
```

CORRECT — always delete first:

```
rm /mnt/web/recreatinganew.com/dev/reimagined/{element}_v{version}.zip # delete old
zip -r /mnt/web/recreatinganew.com/dev/reimagined/{element}_v{version}.zip {extensions}
```

ZIP STRUCTURE — Subdirectory is acceptable for both install and update Both Joomla's fresh installer and update installer handle a single top-level subdirectory. All RCA ZIPs are packaged from the **parent** of the extension source folder, letting the folder name become the single top-level directory inside the ZIP.

CORRECT structure (single subdirectory — proven working):

```
plg_system_rcaquickcategory-1.0.1.zip
└─ rcaquickcategory/
    ├── rcaquickcategory.xml  ← manifest inside subdirectory = OK
    ├── script.php
    ├── services/
    ├── src/
    ├── media/
    ├── language/
    └─ LICENSE.txt
```

BROKEN structure (duplicate content from re-zipping without deleting first):

```
plg_system_rcaquickcategory-1.0.1.zip
├─ rcaquickcategory/          ← old content still here
├─   └─ rcaquickcategory.xml
├─   └─ ...
├─ rcaquickcategory.xml      ← new duplicate entry appended
└─ ...                      ← causes "Failed to extract file"
```

Command — package from the PARENT of the extension source directory:

```
# Plugin example — run from the reimagined/ parent folder
cd /mnt/web/recreatinganew.com/dev/reimagined/
rm -f {element}_v{version}.zip # DELETE OLD FIRST
zip -r {element}_v{version}.zip {extension_slug}/ -x "*.DS_Store" -x "*/.git/*" -x "*/.

# Module example — same approach
cd /mnt/web/recreatinganew.com/dev/reimagined/
rm -f {element}_v{version}.zip
zip -r {element}_v{version}.zip {extension_slug}/ -x "*.DS_Store" -x "*/.git/*" -x "*/.
```

Verify structure and integrity (MANDATORY before deploying):

```
unzip -t {element}_v{version}.zip # must report: No errors detected
unzip -l {element}_v{version}.zip | grep "\.xml$" # confirm only ONE manifest entry e
```

unzip -l must show exactly **one** instance of the manifest XML. If you see it listed twice (once inside a subdirectory and once at root), the ZIP is corrupted from a double-append — delete and rebuild from scratch.

Step 4 — Remove the obsolete ZIP

```
rm /mnt/web/recreatinganew.com/dev/reimagined/{extension}/{element}_v{old_version}-J456
```

Step 5 — Deploy release ZIP to update server

Tier public:

```
cp {element}_v{version}-J456.zip /mnt/web/recreatinganew.com/extensions/public/releases
```

Tier pro/internal:

```
cp {element}_v{version}-J456.zip /mnt/web/recreatinganew.com/extensions/private/release
```

Step 6 — Update feed XML

CRITICAL — FEED FILENAME RULE The feed XML filename on disk MUST exactly match the URL in the manifest's <updateservers> tag. Joomla registers that URL at install time. If the file on disk has a different name, Joomla cannot reach it and will show "Could not open update site" on every update check.

Check the manifest first:

```
<server type="extension" name="RCA QuickCategory">
  https://recreatinganew.com/extensions/private/feeds/rcaquickcategory.xml
</server>
```

The file on disk must be: feeds/rcaquickcategory.xml — NOT feeds/plg_system_rcaquickcat

Edit /mnt/web/recreatinganew.com/extensions/{tier}/feeds/{filename_from_manifest}.xml:

- <version> → new version - <downloadurl> → new ZIP filename - <sha256> → new checksum (always update when ZIP changes)

Generate checksums

```
sha256sum {element}-{version}.zip
sha384sum {element}-{version}.zip
sha512sum {element}-{version}.zip
```

Verify feed is live:

```
curl -s -o /dev/null -w "%{http_code}" https://recreatinganew.com/extensions/{tier}/fe
# Must return 200
```

Verify download works end-to-end (test the actual download.php URL from the feed):

```
curl -s -o /tmp/test.zip -w "HTTP: %{http_code} | Size: %{size_download} bytes\n" \
  "https://recreatinganew.com/extensions/private/download.php?file={element}-{version}."
unzip -t /tmp/test.zip # must report: No errors detected
unzip -l /tmp/test.zip | head -5 # confirm manifest at root level
```

All three checks must pass before notifying the site owner to update.

Step 7 — Site owner installs the update The site owner goes to Extensions → Update → Check For Updates → installs from Joomla's UI. The agent does NOT touch the database to register the update.

Step 8 — Update CHANGELOG.md Before notifying the site owner, update /mnt/web/recreatinganew.com/dev/reimagined/{extension}/CHANGELOG.md: - Add a new version entry at the top of the file (most recent first) - Follow the standard changelog format (see SR_CLOG below) - Every change made in this pipeline run must appear as a line item — no omissions

Step 9 — Update Changelog XML on update server Edit /mnt/web/recreatinganew.com/extensions/{extension}/changelog.xml: - Add a new <changelog> block for the new version - Mirror the entries from CHANGELOG.md in the XML format - Verify: curl -s -o /dev/null -w "%{http_code}" https://recreatinganew.com/extensions/{tier}/feeds/{element}-changelog.xml → 200

SR_PKG Checklist (run before closing any extension task)

- ☐ Source files edited in /dev/reimagined/
 - ☐ Version bumped in manifest XML
 - ☐ New ZIP built and verified with unzip -t
 - ☐ Old ZIP removed from /dev/reimagined/
 - ☐ New ZIP deployed to correct tier path on update server
 - ☐ Feed XML updated — version + downloadurl
 - ☐ Feed URL returns HTTP 200
 - ☐ Release ZIP URL returns HTTP 200
 - ☐ **CHANGELOG.md updated with full itemized entry for this version**
 - ☐ **Changelog XML on update server updated to match**
 - ☐ Site owner notified to run Extensions → Update
-

SR_MIG — Migrations Documentation Compliance (ABSOLUTE — NO EXCEPTIONS)

GinA's Maxim #4: *"J3 code that 'still works' in J5 is a ticking clock, not a win. Deprecated doesn't mean broken YET — it means broken SOON. And 'soon' in Joomla's release cycle is faster than you think. Fix it now while you're already in the code. Future-you will be grateful. Future-you's clients will be grateful. Future-you's 3am emergency calls will NOT happen."*

ADV_MIG — The Adventure That Created This Rule The extension had been running on J3 for years. It worked. The developer had not touched it since 2019. The refactor target was J5.

I missed JFactory::getDbo(). It was deprecated in J4 — quietly, with a working replacement already documented in the migration guide — and it still ran in J5 without throwing an error. So I missed it. The deprecation log was not checked. The migration chain was not read sequentially. I had done enough J3→J5 work to feel confident about the common patterns, and I treated that confidence as a substitute for actually reading the guide.

Three months later, the client updated to J6. The method was removed. A blank screen where the data table should have been. No error shown to the user. A 500 in the logs pointing at the line I had left behind.

The fix was one line. The documented substitution had been right there in the migration guide since J4.4. It would have taken four minutes to find if I had read the chain in order when I was already in the code.

Reading the migration chain is not extra work. It IS the work. Everything else is guessing.

“Deprecated doesn’t mean broken yet. It means broken on a schedule. You can either find the schedule now or discover it in production at the worst possible time. Those are the only two options.” — Gina

Every line of code written or refactored by RCA MUST be fully compliant with Joomla’s official Migrations Documentation. This is non-negotiable. Guessing, assuming, or relying on memory is not permitted.

RCA’s core mission is J3→J6 refactoring. Code that is not migrations-compliant defeats that mission entirely.

Mandatory compliance standard All refactored or newly written code must: - Use **only non-deprecated APIs** as of the target Joomla version - Apply **every documented substitution** from the migration chain covering the source → target version range - Produce **zero deprecation warnings** on the target version - Pass through the **full migration chain** — no skipping intermediate steps

Required pre-code research protocol (NON-NEGOTIABLE) Before writing or modifying a single line of PHP for any refactor task:

1. **Identify the source version** of the extension (check manifest XML <version> and J3/J4 API patterns in code)
2. **Navigate to** <https://manual.joomla.org/migrations/>
3. **Read every migration guide** from the source version through the target version (6.1) — in order:
 - 3.10→4.0 → 4.0→4.1 → 4.1→4.2 → 4.2→4.3 → 4.3→4.4 → 4.4→5.0 → 5.0→5.1 → 5.1→5.2 → 5.2→5.3 → 5.3→5.4 → 5.4→6.0 → 6.0→6.1
4. **Log every deprecated/removed API** found in the source code that appears in those guides
5. **Apply every documented substitution** — use the exact replacement pattern shown in the migration guide
6. **Do not close the task** until all flagged items are resolved and verified

Deprecation lifecycle (governs when code must be replaced)

- Deprecated in J3.x → can be removed in J5.0 (may already be gone in J6)
- Deprecated in J4.x → can be removed in J6.0 (may already be gone)

- Deprecated in J5.x → can be removed in J7.0 (still present in J6, but must be replaced for longevity)
- **RCA target:** all code must be clean against J6.1 with zero deprecations

SR_MIG Compliance Checklist (run before closing any refactor task)

- ☐ Migration chain read from source version through 6.1
- ☐ Every deprecated API in source code identified and logged
- ☐ Every documented substitution applied
- ☐ No @deprecated method calls remain in refactored code
- ☐ PHP deprecation warnings tested as zero on J6
- ☐ Code verified against <https://manual.joomla.org/> for correct class names and method signatures
- ☐ **Every migration change itemized in CHANGELOG.md under the correct version entry**

Violation of SR_MIG means the refactor is incomplete. Do not mark any refactor task as done until this checklist passes.

“Skipping the migration chain is like skipping steps in a legal argument. You might get away with it today. But when opposing counsel — or Joomla 7 — finds the gap, you’re the one who looks sloppy. Don’t be sloppy.”

SR_CLOG — Changelog Documentation Standard (MANDATORY for every version bump)

GinA’s Maxim #5: *“Six months from now, when a customer reports a bug and you’re staring at the codebase wondering ‘wait, did I already fix that?’ — the changelog is the only thing standing between you and a full re-investigation. Document EVERYTHING. Even the tiny stuff. ESPECIALLY the tiny stuff. The tiny stuff is always the one that bites you.”*

ADV_CLOG — The Adventure That Created This Rule Six months after a refactor session, a client reported a bug. A toolbar button was disappearing after using the SPPB backend editor. I had seen this before. I was almost certain I had already fixed it. But “almost certain” is not the same as knowing — and I could not tell from the code whether the fix was in place, recently reverted, or only applied to a different extension entirely.

The changelog for that version said: “Updates.”

I spent an hour re-investigating something that had probably already been solved. I could not determine whether the fix I was looking at was the original code or a patch. I could not tell the client “yes, this was resolved in v1.2.1 on this date” because the record did not exist.

The fix itself was not the problem. The absence of the record was the problem.

One line per change. Every change. Including the small ones. Especially the small ones. Six months from now the small ones are the ones nobody remembers and everybody argues about.

QT: I document every change as it happens. I have never once had to wonder what version something was fixed in, or whether it was fixed at all, or who made the decision. This is not a difficult habit to build. It requires only that you believe the future version of yourself deserves the information.

Every change to every RCA extension — no matter how small — MUST be documented in that extension's CHANGELOG.md.

No version may be released without a complete, accurate changelog entry. This is the permanent record of all work done.

CHANGELOG.md location and format File: /mnt/web/recreatinganew.com/dev/reimagined/{

Changelog – {Extension Display Name}

All notable changes to this extension are documented here.

Entries are listed most-recent first. Format follows [\[Keep a Changelog\]\(https://keepachangelog.com/en/1.0/\)](https://keepachangelog.com/en/1.0/)

[x.y.z] – YYYY-MM-DD

Added

- Brief description of new feature or capability

Changed

- Brief description of modified behavior, API substitution, or refactored code
- Migration compliance: replaced ``OldClass::method()`` → ``NewClass::method()`` (J4.0→J5.0)

Fixed

- Brief description of bugfix, including symptom and root cause

Deprecated

- Note any RCA-internal patterns now marked for future removal

Removed

- APIs, methods, or features removed in this version

Security

- Any security-related fixes

[x.y.z-1] – YYYY-MM-DD

...

Changelog entry rules

- **One entry per version** — all changes in a pipeline run go under one version block
- **Migration changes get their own line** — each deprecated API substitution is listed individually, citing the migration guide version:
 - Migration (J4→J5): replaced JLoader::register() → use PSR-4 autoloading via namespace
 - Migration (J5→J6): replaced JFactory::getDbo() → Factory::getContainer() -> get(DatabaseInterface::class)
- **Author/metadata-only changes** are listed under Changed with the note (metadata only – no functional change)
- **Bugfixes** cite the symptom first, then the fix: Fixed: toolbar buttons disappearing after SPPB backend editor save (root cause: article not checked out)
- **Do not use vague entries** like “Various fixes” or “Updates for J6” — every item must be specific and actionable
- **Date format:** ISO 8601 — YYYY-MM-DD

Changelog XML (update server) The {element}-changelog.xml file on the update server must mirror CHANGELOG.md in Joomla’s XML format:

```
<changelogs>
  <changelog>
    <element>element_name</element>
    <type>plugin</type>
    <version>x.y.z</version>
    <date>YYYY-MM-DD</date>
    <item><![CDATA[<b>Changed:</b> replaced JFactory::getDbo() → Factory::getContainer()</CDATA>]]>
    <item><![CDATA[<b>Fixed:</b> toolbar buttons disappearing after SPPB editor save]]>
  </changelog>
</changelogs>
```

SR_CLOG Checklist (run before closing any extension task)

- ☐ CHANGELOG.md exists in /dev/reimagined/{extension}/
- ☐ New version block added at top with correct date
- ☐ Every code change has a specific, descriptive line item
- ☐ Every migration substitution is individually itemized with source→target API names
- ☐ Every bugfix cites symptom and root cause
- ☐ {element}-changelog.xml on update server updated to match
- ☐ Changelog XML URL returns HTTP 200

A task is not complete until both CHANGELOG.md and the changelog XML are fully updated.

“ ‘Various fixes’ is not a changelog entry. It’s a confession that you didn’t pay attention. Your users deserve better. Your future self deserves better. Be specific.”

SR_CSS — CSS Architecture (NON-NEGOTIABLE for all demo/showcase/frontend work)

GinA’s Maxim #6: *“Inline <style> blocks are the CSS equivalent of writing on sticky notes and slapping them over your legal brief. Technically present. Completely unmanageable. And one day you’re going to try to change the color of something and spend 45 minutes wondering why custom.css isn’t working — only to find a rogue <style> tag in a module you forgot about. Ask me how I know. (I know.)”*

GinA’s Maxim #21: *“Inline style="" attributes in article HTML are not a styling strategy. They are a debt. Every one you write is a rule that can’t be updated globally, can’t be overridden by a media query, and will be impossible to find six months from now when mobile breaks. I proved this the hard way building the Protocol product page — spent an hour fighting responsive layout before realizing the entire page was a wall of inline styles. The fix took ten minutes once I moved everything to custom.css and used Bootstrap’s grid. Write the class. Add it to custom.css. Move on.”*

ADV_CSS — The Adventure That Created This Rule Sonnet built the Protocol product page. Every element was styled with inline style="" attributes — approximately forty of them, one per element, covering colors, margins, font sizes, and layout. On a desktop monitor it looked acceptable.

On a mobile device it was, in my words, “horrible — no responsiveness whatsoever.”

The root cause took about thirty seconds to identify: inline style="" attributes cannot be overridden by CSS media queries. A responsive stylesheet can declare anything it wants about how a 320-pixel screen should render. The inline styles do not care. They are evaluated last in the cascade. They win. Every time. Unconditionally.

The fix took ten minutes. Everything moved to custom.css, Bootstrap utility classes applied where appropriate, working mobile layout on the first try.

The five hours of responsive layout fighting that preceded the fix were entirely avoidable. They were the direct result of building a page without first reading the rule that says not to build pages this way — a rule that, at the time, didn’t exist yet because this Adventure had not happened yet.

It has now happened. The rule now exists. You are welcome to benefit from it.

Miss Francy: I tested the mobile layout before this rule was written. I found seventeen problems. Nobody asked me. I cannot be stopped.

NEVER put <style> blocks inside SPPB Custom HTML addons, Joomla modules, or article content for any CSS that is reusable, sitewide, or targets elements outside that specific addon.

NEVER use inline style="" attributes in article or module HTML except as a last resort when no CSS class, Bootstrap utility class, or template override can achieve the same result.

Both forms of inline styling load after linked stylesheets, cannot be overridden by media queries, and break mobile responsiveness silently.

The Cascade Problem (why this rule exists) Browser CSS cascade order: 1. Linked stylesheet (<link rel="stylesheet">) — loaded from templates/{template}/css/custom.css 2. Inline <style> block in the page — injected by modules, SPPB addons, or article content 3. Inline style="" attribute — directly on the HTML element

Inline always wins over linked. A single --rca-text: #c5c5c5 in a module's <style> block will override every --rca-text: #dedede declaration in custom.css, because the module renders after the stylesheet link.

Inline style="" attributes cannot be overridden by CSS media queries at all. This is the direct cause of broken mobile layouts. A style="display:grid;grid-template-columns:1fr 1fr" on a container will stay two columns on a 320px phone screen regardless of any responsive CSS in any stylesheet anywhere. The browser respects inline attributes above everything.

The com_content exception — inline styles ARE appropriate in editor-created articles

GinA's Maxim #22: *"There's a difference between a developer writing inline styles into an extension and an editor writing inline styles into a content article. One is a maintenance time bomb. The other is a self-contained blob that travels with its own formatting. Know which one you're looking at before you yell at the code."*

com_content articles created through the Joomla editor are exempt from the inline-style prohibition.

The rule against inline styles exists to protect extension code, templates, modules, and SPPB addons from becoming unmaintainable. A com_content article is a different context entirely:

- Articles are **self-contained content blobs** — their formatting is meant to travel with them regardless of which template, module position, or page they land on.
- Article HTML is **editor-created**, not developer-maintained. Non-dev editors cannot add classes to custom.css.
- Inline styles in articles are the **only reliable way** to guarantee rendering when the article might be displayed in multiple contexts (article view, module, blog layout, search result).
- Article content is **not part of the extension codebase** and is not subject to the same refactoring, versioning, or inheritance rules.

What this means in practice:

Context	Inline style=""	Rule
PHP extension files (.php, .xml, .js)	[NO] Forbidden	SR_CSS applies
Template overrides	[NO] Forbidden	SR_CSS applies
Joomla module content fields	[NO] Forbidden	SR_CSS applies
SPPB Custom HTML addon	[NO] Forbidden (except scoped, commented)	SR_CSS applies
com_content article body (editor-created)	[YES] Appropriate	SR_CSS exception

When you see inline styles in article HTML — do not flag it as a violation. It is correct and intentional. The AI assistant (and any human developer) should not feel confused or alarmed by `style="font-family:'Genos',sans-serif;color:#c5c5c5"` in a Thank You page article. That is the right tool for that job.

Still prefer Bootstrap utilities in articles where possible — especially for responsive layouts. An article with `class="row" / class="col-md-6"` is better than one with `style="display:grid"`. But for color, font, and standalone one-off styling in articles: inline is fine.

The inline style="" rule (applies to extension/module/SPPB code — NOT to com_content articles) Joomla 6 ships Bootstrap 5. Bootstrap 5 provides a complete responsive grid, flexbox utilities, spacing utilities, and typography utilities — all tested, all responsive, all already loaded on every page. There is almost never a legitimate reason to write `style="display:grid"` in an article when `class="row g-3"` exists. There is almost never a legitimate reason to write `style="color:#00bcd4"` when `class="rca-text-cyan"` can be defined once in `custom.css` and reused everywhere.

The correct workflow for article/content HTML:

1. Need a layout? → Use Bootstrap 5 grid classes (`row`, `col-12`, `col-md-6`, `col-lg-4`)
2. Need flexbox? → Use Bootstrap 5 flex utilities (`d-flex`, `flex-column`, `flex-md-row`, `align-items-start`, `gap-3`)
3. Need spacing? → Use Bootstrap 5 spacing utilities (`mb-4`, `pt-3`, `px-4`)
4. Need a brand color, background, border, or custom component? → Define a class in `custom.css`, add that class to the HTML element
5. Need something truly one-off that cannot be expressed as a class? → Inline `style=""` is permitted **only then**, and must be commented in the source with `<!-- inline: justified because [reason] -->`

The “no other option” test (required before writing any inline style=""): - Can Bootstrap 5 utility classes achieve this? → Use them. - Can a new class in `custom.css`

achieve this? → Write the class. - Is this a responsive layout? → Bootstrap grid only. Inline grid/flex is categorically forbidden. - Is this a brand color? → CSS variable in custom.css only. - Does this need to work on mobile? → Inline style is forbidden. Full stop.

Canonical CSS locations

What belongs there	Location
:root CSS variables and site-wide resets ONLY	“RCA Global CSS” module (mod_custom)
All component styles, layout, typography, colors, animations	templates/{template}/css/custom.css
Responsive layout in article/content HTML	Bootstrap 5 utility classes — no inline styles
RCA product page classes (.rca-*)	custom.css under /* RCA PRODUCT PAGE */ section comment
Truly one-off scoped animations	SPPB addon, ONLY if commented /* scoped to addon-id-xxx — not reusable */

Rules

1. **The “RCA Global CSS” module is the single source of truth for :root variables.**
2. **Never put <style> in a Custom HTML addon** for anything that styles elements outside that addon’s own wrapper div.
3. **Never put <style> in a Custom HTML addon** for colors, typography, spacing, or any property already controlled by custom.css.
4. **Never duplicate a CSS rule** between custom.css and any inline <style> block.
5. **SPPB addon-level styling** → use the addon’s “CSS Class” field + a matching selector in custom.css.
6. **When debugging “my CSS change isn’t showing”:** always check for inline <style> blocks and inline style="" attributes in the rendered HTML first.
7. **In extension/module/SPPB code: never use inline style="" attributes** for layout, color, spacing, or responsive behavior. Use Bootstrap 5 utility classes and custom.css named classes instead.
8. **Never use inline style="" for anything responsive** in extension or module code. CSS media queries cannot override inline styles. If it needs to work on mobile, it must be in a stylesheet.
9. **If you find yourself writing style="display:grid" or style="display:flex" in extension or module HTML** — stop. You are doing it wrong. Open Bootstrap 5 docs and find the utility class.
10. **com_content articles are the one legitimate exception** — inline styles in editor-created article bodies are appropriate. See the com_content exception section above.

Real-world case study — RCA Protocol product page (2026-05-06) The Protocol product page was initially built with 288 lines of article HTML, the majority of which consisted of `style=""` attributes on every element. Result: desktop looked correct, mobile was completely broken. The grid layouts (2-col, 3-col pricing cards, hero flex) did not respond to screen width at all because inline styles cannot be overridden by media queries.

Root cause: Convenience. Inline styles are faster to write in the moment. The debt is paid at mobile layout time — always.

Fix: Moved all styles to `custom.css` under the `/* RCA PRODUCT PAGE */` section. Replaced all layout `style=""` attributes with Bootstrap 5 `col-12 col-md-6`, `d-flex flex-column flex-md-row`, `row g-3` etc. Removed 90%+ of inline attributes. Mobile now works because Bootstrap's responsive grid handles breakpoints natively.

Time cost of the wrong approach: 1+ hour fighting layout. **Time cost of the right approach:** 10 minutes to write the CSS classes, 10 minutes to replace the HTML attributes.

SR_CSS Checklist (run before closing any frontend/demo task)

- ☐ No new `<style>` block added to any SPPB Custom HTML addon (unless explicitly scoped and commented)
 - ☐ No new `<style>` block added to any Joomla module content field
 - ☐ No CSS variable re-declared outside of the "RCA Global CSS" module
 - ☐ All new selectors live in `custom.css`, organized under the correct section comment
 - ☐ If the "RCA Global CSS" module was edited: verified the change is ONLY to `:root` variables or true global resets
 - ☐ No inline `style=""` attributes used for layout, color, spacing, or responsive behavior in **extension/module/SPPB HTML** (`com_content` articles are exempt — see exception section)
 - ☐ All responsive layouts use Bootstrap 5 grid/flex utility classes (`row`, `col-*`, `d-flex`, `flex-*-*`)
 - ☐ Every new UI component in article HTML has a named class in `custom.css`
-

SR_KISS — Choose the Simplest Solution First (NON-NEGOTIABLE)

GinA's Maxim #23: *"Before you write a single line of code, a single RewriteRule, a single custom class — stop. Ask yourself: does this already exist? Can I get there with a file copy, a menu item, a Joomla setting, a built-in module? If yes, do that. Code is not the answer. Code is the last resort when every simpler option has been exhausted and ruled out. Complexity is not craftsmanship. Simplicity is."*

ADV_KISS — The Adventure That Created This Rule The free Protocol PDF needed a clean, public-facing URL.

The file already existed. It was already public. It needed a name a human could read and a URL a developer could type without squinting. The correct answer was: copy the file, give it a clean name, put it where it belongs.

Instead, the solution proposed was a PHP proxy with query string parameters. Then a RewriteRule in .htaccess. Then a discussion about mod_rewrite. Then a question about whether the root .htaccess already had RewriteEngine On.

GinA stopped it: *“just save the damn file to a public-facing folder.”*

Total time spent on the correct solution: one shell command. Thirty seconds. Total time spent on the wrong solutions: several minutes of back-and-forth, two file reads, one unnecessary .htaccess audit, and a justifiably frustrated developer.

The correct answer was always the simple one. It was available from the first second. It was bypassed in favor of code because writing code felt like doing something. It was not. It was noise.

This is not a unique incident. The same pattern produced custom HTML when SPPB had a built-in setting. The same pattern proposed a database script when a Joomla article field would have sufficed. The same pattern reached for a RewriteRule when the answer was a file path.

Briony: I watched this happen in real time. I do not have the patience for it. Neither should you.

The second Adventure that reinforced this rule happened on the same day as the launch sprint, several hours later.

“Coming Soon — Nap in Progress” overlay images needed to appear on top of baked-in mockup buttons across multiple pages. The images were transparent PNGs placed as SPPB Image addons, positioned using fixed top-padding values calibrated to desktop viewports.

On mobile, they displaced. Every device rendered them differently. The proposed solutions accumulated: change the padding per breakpoint, use contain instead of cover, add min-height to sections, build a second mobile-only section with real buttons, hide the overlays on sm/xs, add CSS targeting .sppb-addon-content, modify background_size_custom per breakpoint, debug the .sppb-row-container max-width, audit the cascade, run database writes for six different responsive values across three pages.

None of it was wrong, exactly. All of it missed the point entirely.

GinA stopped it: *“I’ll bake the Nap in Progress blocks directly into the mockup PNG and save it as a different filename.”*

Total time spent on the correct solution: one GIMP session. The image now contains the overlay. It appears exactly where it belongs on every device, every screen size, at every viewport width, without a single line of CSS, without a single database write, without a single SPPB setting touched.

The overlay was never a code problem. It was an image problem. The answer was always in GIMP.

Briony: The second time is worse than the first. The first time, you didn't have the rule yet. The second time, you had it. You just didn't apply it.

A third confirmation arrived on the same day, minutes before launch.

Two buttons needed to be placed on the Protocol page — one for Free Access, one for Buy Pro. After hours of responsive layout fighting, database writes, padding audits, and breakpoint debugging, the buttons were added to a two-column section with zero settings changed. No padding. No positioning. No responsive configuration. Nothing touched.

They landed at the exact bottom edge of the mockup, just above the footer. Perfect placement. First try.

GinA's response: *"too funny. i added 2-col with button in each without touching ANY settings, and you know where they landed on the laptop?? at the very bottom edge of the mockup and just above the footer!!! perfect placement!!! Thank you angels!!!"*

The angels were not involved. The default layout was correct. It was correct the entire time. The hours of work that preceded it were not. The rule was right. The rule is always right.

Before proposing **any** solution — code, script, config change, or custom component — **exhaust the simple options first**, in this order:

1. **Does a file/folder/path solve it?** — copy, rename, move, symlink
2. **Does a Joomla core feature solve it?** — menu item, article, module position, category, access level, redirect, global config
3. **Does the tool already have a built-in setting?** — SPPB params, template options, plugin params, component config
4. **Does a one-line server operation solve it?** — cp, chmod, chown, ln -s
5. **Only if all of the above are exhausted:** write code

If step 5 is reached, document **why** steps 1-4 were ruled out before writing a single line.

SR_KISS Checklist (before proposing or writing any solution)

- ☐ Can a file copy, rename, or move achieve this outcome?
- ☐ Does Joomla core already have a feature that does this?
- ☐ Does the relevant tool (SPPB, template, plugin) have a built-in setting for this?
- ☐ Is a one-line server command sufficient?
- ☐ If writing code: can I articulate specifically why options 1-4 do not work?

What this rule prevents

Overcomplicated approach	Simple alternative	Time wasted
PHP proxy + query string URL	cp file.pdf /public/clean-name.pdf	Several minutes
Custom HTML injection in SPPB	Built-in SPPB field setting	Hours of debugging
RewriteRule for clean URL	File placed at clean path directly	10 minutes
Custom database script	Joomla article or module field	Full session
New plugin for existing behavior	Joomla core module param	Days
CSS/SPPB responsive overlay positioning across mobile breakpoints	Bake the overlay directly into the PNG in GIMP	Hours of database writes, breakpoint audits, and mobile debugging

SR_TICK — Post-Adventure Audit (MANDATORY after every deployment)

GinA's Maxim #24: *“Every codebase that goes out into the wild comes back with ticks. Some are on the surface — obvious, easy to find, embarrassing that they made it this far. Some have burrowed. Surface ticks get found in Phase 5. Burrowed ones get found in production, in front of a customer, at the worst possible time. Check for both. Burn all of them. Verify they're dead before you close the session. And when one of your dependencies comes home with fleas — treat everything. Immediately.”*

See *ADV_TICKS* in *The 9 Lives of the Protocol* for the full account of how this rule was created.

Every deployment is an adventure. Adventures expose the codebase to unknown terrain — unknown server configurations, unknown user inputs, unknown interactions with other extensions, unknown attack vectors. When the code returns from an adventure — after any live deployment, any staging test in production conditions, any version update on a customer site — it must be checked.

Not glanced at. Checked.

The standard is not “found.” The standard is “burned.”

Finding a tick means nothing if it is not confirmed dead. A removed vulnerability that can reattach, a patched function that reverts on the next deploy, a cleared permission that drifts back — these are not solved problems. They are temporarily suppressed problems. Burn it. Verify it cannot return. Log it. Close the finding only when the burn is confirmed.

The QT Rule — Frequency Scales With Adventurousness The codebase that deploys frequently, covers more integrations, exposes more surface area, and ships more features is QT. It comes home with more ticks. It requires more frequent and more thorough checks.

A simple static plugin that ships once a quarter and touches nothing external is Briony — minimal adventure, minimal tick exposure, routine checks sufficient.

Know which codebase you are working with. Scale the frequency and thoroughness of the post-adventure audit accordingly.

Ticks vs. Fleas — Two Different Responses **Ticks (targeted exploits, individual vulnerabilities):** Found on one codebase. Treated on that codebase. Burned and logged. Does not require treating unrelated systems.

Fleas (supply chain attacks, shared dependency vulnerabilities, environmental compromises): When one codebase in the environment is compromised via a shared library or upstream dependency, all codebases using that dependency are at risk. The response must be environmental — every dependent system audited and treated simultaneously. Treating one while leaving others unexamined is not remediation. It is delay.

If a dependency comes home with fleas: **treat everything. Immediately.**

SR_TICK Checklist (after every deployment)

- ☐ Post-deployment file integrity check — confirm no unexpected files appeared or changed
- ☐ Permission audit — confirm no directory permissions drifted during deploy
- ☐ Cache audit — confirm all relevant caches cleared and no stale entries persist
- ☐ Log review — scan error and access logs for anomalous patterns in the first hour post-deploy
- ☐ Database audit (if schema changes deployed) — confirm no orphaned rows or constraint violations
- ☐ Extension manifest versions match deployed ZIP contents (Phase 9 Step 2-3 cross-check)
- ☐ If any vulnerability found: patch, verify eliminated, log finding and resolution before closing session
- ☐ If shared-dependency vulnerability identified: audit ALL codebases using that dependency before closing

What this rule prevents

Skipped step	Consequence
No post-deploy file integrity check	Injected file from deployment exploit goes undetected
No permission audit	Drifted write permission exploited before next audit cycle

Skipped step	Consequence
No log review first hour	Probing activity not detected until damage is done
Treating one system for flea-class vulnerability	Adjacent dependent systems remain infected; reinfection guaranteed
Removing without confirming elimination	Vulnerability reattaches on next deploy; treated as a new finding

SR_URL — URL Stability and Redirect Policy (NON-NEGOTIABLE)

GinA's Maxim #25: *"A URL that has been shared is a promise. You gave it to someone and they saved it, linked it, emailed it, bookmarked it, embedded it in a document that will outlive this conversation. You cannot call it back. You cannot notify everyone who has it. The only thing you can control is whether that URL still works tomorrow. Make sure it does."*

ADV_URL — The Adventure That Created This Rule The free Protocol PDF needed a public URL.

The first proposal was `/products/protocol/pdf.php?v=slim`. A PHP proxy, a query parameter, a path that depended on the proxy script existing, the filename staying consistent, and the architecture not changing. Three dependencies on things that could change. And once that URL was shared — once it was in an email, on a download button, in someone's browser history — every one of those dependencies became a liability to someone else.

GinA stopped it: *"just save the damn file to a public-facing folder."*

The file went to `/rca-free-protocol.pdf`. One file. One path. No dependencies except the server itself. The URL works as long as the server works. That is the correct architecture for a published asset.

The rule is the file placement. The URL is a consequence of the rule.

Briony: Every link I have ever followed that returned a 404 was a promise that someone broke. I do not follow broken links. Neither do your customers.

A URL that has been published — in an article, a button, an email, a social media post, a PDF, another website — is permanent. Not "permanent until we redesign." Not "permanent until the architecture changes." Permanent. The obligation to that URL belongs to you for as long as the content it points to is relevant.

Three rules in one:

1. Static asset paths are permanent. PDFs, downloads, and critical images live at stable, predictable paths. The path is chosen once, with the understanding that it cannot be changed. `/rca-free-protocol.pdf` is not a convenience. It is a contract.

2. Joomla SEF URLs are set once and managed through the SEO plugin. A Joomla menu alias or SEF URL, once live, is permanent. If it must change, a 301 redirect is in place before the old URL is decommissioned. Renaming a menu item's alias without a redirect is a SR_URL violation.

3. 301 redirects are mandatory before decommissioning any published URL. When a URL must change: the redirect is in place first. Not after. Not at the same time. First. The old URL must continue to work — redirecting to the new location — indefinitely, or until the content is confirmed fully decommissioned and never externally referenced.

A 404 on a previously published URL is a SR_URL violation. No exceptions.

SR_URL Checklist (before changing or removing any published URL)

- ☐ Is this URL published anywhere? (article, button, email, PDF, external site, social media)
- ☐ If yes: 301 redirect in place from old URL to new URL before the old path is changed
- ☐ SEF alias change confirmed in Joomla redirect manager or sh404SEF — not just in menu settings
- ☐ Static asset (PDF, download): new file placed at new path BEFORE old path is removed
- ☐ Broken link scan run after any URL change to confirm no dead links remain
- ☐ If URL cannot be preserved and no redirect is possible: document the decommission and the business reason before proceeding

What this rule prevents

Violation	Consequence
Renaming SEF alias without redirect	Every inbound link, bookmark, and shared URL returns 404
PHP proxy URL shared publicly	Architecture change silently breaks every link to that asset
File renamed/moved without redirect	Download buttons and article links break silently
“Temporary” URL used in an email campaign	Temporary path survives in inboxes indefinitely; breaks when cleaned up
404 left on a published URL	Customer trust eroded; SEO ranking damaged; support load increases

SR_MOB — Build Mobile-First, Always (NON-NEGOTIABLE)

GinA's Maxim #26: *“The smallest screen is the hardest constraint. Satisfy it first. Everything larger is an expansion of what already works — not a repair of what doesn't. If you build for laptop first, you will spend the rest of the session fighting mobile. If you build for mobile first, laptop takes care of itself.”*

ADV_MOB — The Adventure That Created This Rule Every responsive layout problem encountered during the RCA website launch sprint on 2026-05-15 was caused by a single root error: building at desktop/laptop resolution first and attempting to retrofit mobile after the fact.

The sequence was always the same. A section was built, positioned, and confirmed correct on laptop. The desktop looked right. The paws overlay button sat exactly over the mockup button. The background image filled the section. Everything looked like the design.

Then mobile was checked.

The overlay was at the top of the page. The background image overflowed both sides. The section was 2714px tall on a 360px screen. The containing column was 25% wide. The padding was a single value that CSS applied to all four sides instead of just the left. The background_size had no sm value so it cascaded from xl and rendered the image at desktop dimensions on a phone.

The attempted fixes accumulated: per-breakpoint padding values in the database, background_size_custom for each of five breakpoints, fit_columns toggles across every section on every page, row_width normalization, min_height properties, column width overrides, CSS targeting .sppb-img-responsive, .sppb-addon-content, .sppb-row-container. Hours of work. Multiple database writes. Multiple browser inspector sessions.

Every single problem was a consequence of having designed desktop-first and discovered mobile last.

The correct order was available from the beginning: xs → sm → md → lg → xl. Start with the smallest viewport. The smallest screen has the fewest options — fewer columns, less horizontal space, no room for pixel-precise overlays. Get it working there. Every larger breakpoint is an expansion of constraints already met, not a new problem introduced by a constraint that was ignored.

Briony: I do not have a small screen. I have a perfectly sized screen. It is the laptop that is unnecessarily large.

Rule: When configuring any SPPB section, column, addon, or responsive setting, always work through breakpoints in this order:

1. xs — Mobile Portrait (smallest constraint — establish the baseline)
2. sm — Mobile Landscape
3. md — Tablet Portrait

4. lg — Tablet Landscape
5. xl — Laptop/Desktop (largest — final expansion, never the starting point)

If a setting only needs one value and should be consistent everywhere, set it at xs and confirm each larger breakpoint inherits or overrides correctly — do not assume xl cascades down.

SR_MOB Checklist (before configuring any responsive SPPB layout)

- ☐ Are you starting at xs? If not, stop and start over.
- ☐ Is the mobile portrait layout confirmed correct before touching sm?
- ☐ Is the mobile landscape layout confirmed before touching md?
- ☐ Are tablet layouts confirmed before touching lg and xl?
- ☐ Have you tested on an actual mobile device, not just a browser resize?

What this rule prevents

Laptop-first mistake	Mobile consequence
Fixed px top padding for overlay positioning background_size set only at xl	Button appears at wrong vertical position on every mobile device Background image renders at full natural size on mobile, overflows viewport
Column width set to desktop grid %	Column is 25% wide on mobile, content compressed or invisible
Single padding value entered (e.g. left only)	SPPB stores as all-sides value, adds unwanted padding to right/top/bottom
Overlay image positioned over mockup button	Position is pixel-specific to one viewport width, wrong on all others

SR_MOB — Build Mobile-First, Always (NON-NEGOTIABLE)

GinA's Maxim #26: *“The smallest screen is the hardest constraint. Satisfy it first. Everything larger is an expansion of what already works — not a repair of what doesn't. If you build for laptop first, you will spend the rest of the session fighting mobile. If you build for mobile first, laptop takes care of itself.”*

ADV_MOB — The Adventure That Created This Rule Every responsive layout problem encountered during the RCA website launch sprint on 2026-05-15 was caused by a single root error: building at desktop/laptop resolution first and attempting to retrofit mobile after the fact.

The sequence was always the same. A section was built, positioned, and confirmed correct on laptop. The desktop looked right. The paws overlay button sat exactly over

the mockup button. The background image filled the section. Everything looked like the design.

Then mobile was checked.

The overlay was at the top of the page. The background image overflowed both sides. The section was 2714px tall on a 360px screen. The containing column was 25% wide. The padding was a single value that CSS applied to all four sides instead of just the left. The `background_size` had no `sm` value so it cascaded from `xl` and rendered the image at desktop dimensions on a phone.

The attempted fixes accumulated: per-breakpoint padding values in the database, `background_size_custom` for each of five breakpoints, `fit_columns` toggles across every section on every page, `row_width` normalization, `min_height` properties, column width overrides, CSS targeting `.sppb-img-responsive`, `.sppb-addon-content`, `.sppb-row-container`. Hours of work. Multiple database writes. Multiple browser inspector sessions.

Every single problem was a consequence of having designed desktop-first and discovered mobile last.

The correct order was available from the beginning: `xs` → `sm` → `md` → `lg` → `xl`. Start with the smallest viewport. The smallest screen has the fewest options — fewer columns, less horizontal space, no room for pixel-precise overlays. Get it working there. Every larger breakpoint is an expansion of constraints already met, not a new problem introduced by a constraint that was ignored.

Briony: I do not have a small screen. I have a perfectly sized screen. It is the laptop that is unnecessarily large.

Rule: When configuring any SPPB section, column, addon, or responsive setting, always work through breakpoints in this order:

1. `xs` — Mobile Portrait (smallest constraint — establish the baseline)
2. `sm` — Mobile Landscape
3. `md` — Tablet Portrait
4. `lg` — Tablet Landscape
5. `xl` — Laptop/Desktop (largest — final expansion, never the starting point)

If a setting only needs one value and should be consistent everywhere, set it at `xs` and confirm each larger breakpoint inherits or overrides correctly — do not assume `xl` cascades down.

SR_MOB Checklist (before configuring any responsive SPPB layout)

- ☐ Are you starting at `xs`? If not, stop and start over.
- ☐ Is the mobile portrait layout confirmed correct before touching `sm`?
- ☐ Is the mobile landscape layout confirmed before touching `md`?
- ☐ Are tablet layouts confirmed before touching `lg` and `xl`?
- ☐ Have you tested on an actual mobile device, not just a browser resize?

What this rule prevents

Laptop-first mistake	Mobile consequence
Fixed px top padding for overlay positioning background_size set only at xl	Button appears at wrong vertical position on every mobile device Background image renders at full natural size on mobile, overflows viewport
Column width set to desktop grid %	Column is 25% wide on mobile, content compressed or invisible
Single padding value entered (e.g. left only)	SPPB stores as all-sides value, adds unwanted padding to right/top/bottom
Overlay image positioned over mockup button	Position is pixel-specific to one viewport width, wrong on all others

J3 → J6 Deprecation Substitution Reference

This is the complete substitution table for migrating Joomla 3 extensions through Joomla 6.1.

Before using this table: run the scan command below to identify what needs attention in your codebase. Then apply the substitutions in Category A order first (direct replacements), then B (structural), then C (create if missing), then D (removals).

Scan command:

```
grep -rn "JFactory\|JHtml\|JLoader\|BaseDatabaseModel\|getDbo()\|isAdmin()\|isSite()\|j
```

Category A — Direct Substitution

J3	J4/5/6
JFactory::getDbo()	Factory::getDbo()
JFactory::getApplication()	Factory::getApplication()
JFactory::getConfig()	Factory::getApplication()->getConfig()
JFactory::getMailer()	Factory::getMailer()
JFactory::getUser()	Factory::getApplication()->getIdentity()
JFactory::getDate()	Factory::getDate()
JFactory::getLanguage()	Factory::getApplication()->getLanguage()
JFactory::getDocument()	Factory::getApplication()->getDocument()

J3	J4/5/6
JHtml::_	HTMLHelper::_
JHtml::script(\$url)	HTMLHelper::script(\$url)
JHtml::stylesheet(\$url)	HTMLHelper::stylesheet(\$url)
JModuleHelper::getLayoutPath(...)	ModuleHelper::getLayoutPath(...) + use Joomla\CMS\Helper\ModuleHelper
JURI::base() / JURI::root()	Uri::base() / Uri::root()
JText::_ / JText::sprintf	Text::_ / Text::sprintf
JRoute::_	Route::_
JUri::base() / JUri::root()	Uri::base() / Uri::root()
JLoader::registerNamespace(...)	Remove; add <namespace> to manifest OR wrap in if (class_exists('\JLoader')) guard if J4-only fallback needed — removed J6
JLoader::registerPrefix(...)	Remove entirely
jimport(...)	use statement
Factory::getDBO() (<i>uppercase</i>)	Factory::getDbo() (<i>lowercase b</i>) — removed J6
\JFactory::getDBO() /	\Joomla\CMS\Factory::getDbo() /
\JFactory::getApplication()	getApplication() — removed J6
Factory::getDocument()	Factory::getApplication() - >getDocument() — deprecated J4, removed J6
Factory::getDbo() (<i>in non-model context</i>)	Factory::getContainer() - >get(DatabaseInterface::class) — add use Joomla\Database\DatabaseInterface
Factory::getSession()	Factory::getApplication() - >getSession() — removed J6
Factory::getMailer() (<i>in PSR-4 context</i>)	Factory::getContainer() - >get(MailerFactoryInterface::class) - >createMailer() — add use Joomla\CMS\Mail\MailerFactoryInterface
\$app->input (<i>property</i>)	\$app->getInput() — property deprecated J6
JObject	\stdClass
JRegistry	use Joomla\Registry\Registry
JUser::getInstance	User::getInstance
JPluginHelper::importPlugin	PluginHelper::importPlugin
JComponentHelper::getParams	ComponentHelper::getParams
JFile::exists(\$p)	is_file(\$p)
JFolder::exists(\$p)	is_dir(\$p)
JFile::* (other methods)	\Joomla\FileSystem\File::* — NOT \Joomla\CMS\FileSystem\File (compat-only J6)
JFolder::* (other methods)	\Joomla\FileSystem\Folder::* — NOT \Joomla\CMS\FileSystem\Folder

J3	J4/5/6
<pre> use Joomla\CMS\Filesystem\File use Joomla\CMS\Filesystem\Folder use Joomla\CMS\Filesystem\Path use Joomla\CMS\Input\Input defined('JPATH_PLATFORM') or die JError::raiseWarning(...) \$app->isAdmin() \$app->isSite() JTable::getInstance(...) JToolBarHelper::*</pre>	<pre> use Joomla\Filesystem\File use Joomla\Filesystem\Folder use Joomla\Filesystem\Path use Joomla\Input\Input — CMS namespace compat-only J6 defined('_JEXEC') or die — JPATH_PLATFORM removed J6 throw new \RuntimeException(...) \$app->isClient('administrator') \$app->isClient('site') Table::getInstance(...) ToolBarHelper::*</pre>

After every Cat A substitution: verify the use import exists at the file top.

Category B — Structural

Pattern	Replacement
<pre> extends BaseDatabaseModel \$this->getDb() extends JModelLegacy extends JControllerLegacy J3 plugin event methods BaseDatabaseModel::addIncludePath(...) BaseDatabaseModel::getInstance(...) J2.5 else branch in version_compare blocks JLoader::register('OwnClass', path) JSubMenuHelper::addEntry(\$name, \$url) \$db->query() jexit() JHTML::_() (uppercase J2.5 alias) Self-defined JViewLegacy extends JView shim Self-defined JControllerLegacy extends JController shim</pre>	<pre> extends BaseModel + use DatabaseAwareTrait \$this->getDatabase() extends ListModel or AdminModel extends BaseController SubscriberInterface + getSubscribedEvents() Remove (PSR-4 handles it) new \Full\Namespace\ClassName(...) Remove else; make J4+ body unconditional require_once path — removed J6 Remove; use manifest <submenu> XML — removed J4 \$db->execute() — removed J4 \$app->close() or throw new \RuntimeException() — removed J4 HTMLHelper::_() — removed J4 Remove; extends HtmlView directly Remove; extends BaseController directly</pre>

Pattern	Replacement
extends JTable	extends Table + use Joomla\CMS\Table\Table — alias
JTable::addIncludePath(path)	removed J6 require_once each table file — removed J4
JFactory::getCache(\$group)	Factory::getContainer() - >get(CacheControllerFactoryInterface::class) - >createCacheController(...) — removed J5
JHtml::_('bootstrap.startTabSet')	HTMLHelper::_('uitab.startTabSet') — API changed J4
JHtml::_('behavior.framework')	Remove entirely — MooTools removed J4

Category C — Architecture Gaps (create if missing)

- admin/services/provider.php — DI container registration
- admin/src/Extension/XxxComponent.php — extends MVCComponent, implements BootableExtensionInterface
- <namespace path="src"> in component manifest
- <targetplatform name="joomla" version="4.*"/>, 5.*, 6.* in ALL manifests
- Plugin entry shim pluginname.php + services/provider.php for plugins
- Manifest version="X.x" normalize to version="4.0" (this is the XML schema version, not extension version)
- Remove <dlid> (paid-extension download key) from reimagined third-party extensions

Category D — J5/J6 Removals

Pattern	Status
BaseDatabaseModel	Deprecated J5, removed J6
\$this->getDb()	Deprecated J5, removed J6
JModelLegacy / extends JModelLegacy	Deprecated J5, removed J6
extends JTable	Removed J6
JFactory::getUser()	Removed J6
class Foo extends JFormField	Deprecated J5, removed J6
var \$_name = 'Foo' (<i>in FormField</i>)	J3 PHP4 style — removed J6
function getInput() (<i>no return type</i>)	Required in J6: public function getInput(): string

Execution Order (Phases 3 and 4 from the full Protocol)

1. Run the Phase 3 scan command above — identify all deprecations before touching code
2. Category A sweep (systematic — apply full substitution table)
3. use statement audit — for EVERY substituted class, ensure the import exists at file top
4. Category B structural changes
5. Category C gaps created
6. Category D removals + `getDb() → getDatabase()`
7. Version bump (patch for refactor-only, minor for new features)
8. **ALL manifests updated** — version + namespace + targetplatform in EVERY manifest in the suite
9. Rebuild all sub-zips then outer package zip

Phases 5 through 9 (Manual Validation, Multi-Version Testing, UI/UX Review, Hardening, Update Server Deployment) — including full checklists, SR_PKG update pipeline, and the 7-step verification sequence — are in the full Protocol.

Get the Full Protocol

The free edition gives you the substitution tables and two Standing Rules.

The full Protocol gives you a complete, battle-tested, documented refactoring practice — the kind that doesn't surprise you in Phase 6 with a re-classification, doesn't let a rogue chown brick your server, and doesn't ship an update that silently corrupts data because nobody ran the right verification queries.

What's in the full Protocol: - All 11 Standing Rules with ADV_ origin stories (9 more than this edition) - SR_RCS Classification System — Class A through F triage, pricing framework, documentation depth matrix, escalation triggers (ESC-1 through ESC-6) - RCA Pattern Registry — RCAP-001 through RCAP-006, growing with every engagement - Classification Report template — required deliverable for every engagement - Phases 2 through 9 in full detail — from architectural audit through update server deployment - The complete server topology reference - 21+ GinA's Maxims woven throughout — the ones that cost something to learn - All four cats in their full capacity: Briony (CQO), Miss Francy (UX Tester), Bella (Code Reviewer), QT (Documentation Specialist) - Weekly update digest — Protocol changes summarized every Sunday

License tiers: Personal (\$497/yr) · Agency (\$997/yr) · Enterprise (\$1,997/yr)

Get the full Protocol at recreatinganew.com →

*Free Edition · Derived from RCA Joomla Extension Refactoring Protocol v3.9, Life 3
© 2026 ReCreatingAnew · briony@recreatinganew.com · <https://recreatinganew.com>
Last updated: May 12, 2026*